

Fault Injection Evaluation with Statistical Analysis

How to Deal with Nearly Fabricated Large Circuits

Felix Uhle¹, Nicolai Müller¹ and Amir Moradi²

¹ Horst Görtz Institute for IT Security, Ruhr-Universität Bochum, Bochum, Germany

firstname.lastname@rub.de

² Technische Universität Darmstadt, Darmstadt, Germany

firstname.lastname@tu-darmstadt.de

Abstract. A critical aspect of securing cryptographic hardware is their resistance to Fault Injection (FI) attacks, which involve the successful injection of faults into the system in operation. Specifically, a hardware design must be resilient to well-established fault injection techniques, including voltage or clock glitching, laser fault injections, and the more recently introduced Electromagnetic Fault Injection (EMFI). Ideally, the protection level must be verified before the chip is fabricated. Although initial efforts to verify the resistance of hardware designs against fault injection have been made, analyzing the security of practical designs with realistic gate counts under fault injections that affect multiple gates or the entire circuit state remains a significant challenge. This scenario, however, is considered more realistic than assessing resistance to a fixed, relatively small number of faults. In this work, we introduce FIESTA, a versatile automated framework for analyzing the resistance of hardware circuits under the general random fault model. By leveraging a non-exhaustive approach, FIESTA is capable of evaluating larger designs compared to state-of-the-art tools, while maintaining a reasonable level of confidence. FIESTA supports various adversary models, allowing customized resistance analysis against specific adversaries. In particular, we present a concrete procedure for evaluating more realistic precise adversaries, based on practical observations. Using FIESTA, we assessed the resistance of several (protected) Advanced Encryption Standard (AES) cores.

Keywords: Fault Injection, Hardware, Random Fault Model, General Random Fault Model

1 Introduction

Fault injection attacks, in which an adversary deliberately manipulates a device to force it into incorrect behavior, remain a significant threat to the security of cryptographic implementations. In their seminal work, Boneh *et al.* [BDL97] introduced a theoretical fault attack on the asymmetric RSA cryptosystem, while related Differential Fault Analysis (DFA) attacks are applicable to nearly all symmetric ciphers and highly effective in practice [BS97]. Over the years, advanced fault attacks such as Ineffective Fault Analysis (IFA) [Cla07], Fault Sensitivity Analysis (FSA) [LSG⁺10], Statistical Fault Analysis (SFA) [FJLT13], and the more recent Statistical Ineffective Fault Analysis (SIFA) [DEK⁺18] have been introduced. Notably, SIFA is particularly powerful due to its significantly relaxed requirements regarding the timing, location, and type of injected faults, as well as its reduced reliance on specific fault models compared to the similar attacks. In practice, this simplifies the fault injection process, eliminating the need for highly precise methods, such as inducing faults through focused laser beams [SA02]. Instead, EMFI, which disrupts

a device using a strong electromagnetic pulse, has gained increasing attention [QS01]. Among its key advantages over laser-based fault injection is its typically lower equipment costs and the ability to induce faults on a chip without decapsulation, meaning the package remains intact [DLM19]. To counteract fault injection attacks, particularly those enabled by the widely applicable EMFI, several countermeasures have been developed. These countermeasures aim to detect or entirely correct a predefined number of up to k injected faults by employing redundancy, either through direct duplication of sensitive hardware components or through the use of binary codes [AMR⁺20, SRM20, RSM21]. The underlying adversary model, known as the threshold fault model, defines an adversary capable of faulting at most k gates of a circuit [IPSW06]. The threshold fault model is primarily designed to facilitate security proofs, including automated ones [AWMN20, RSS⁺21], while capturing the impact of highly precise faults, such as those induced by a focused laser beam. However, practical experiments reported in [BBM⁺22] show that even a single laser pulse often affects multiple gates simultaneously. This is due to the relatively low transistor density in modern hardware chips, calling into question the relevance of the threshold fault model. With transistor densities expected to shrink further – the 2 nm technology node is already slated for 2025 – and the much broader impact range of EMFI compared to laser fault injection, modern adversary models must evolve to account for faults that usually affect multiple gates simultaneously. As a natural reaction to this, the random fault model assumes an adversary capable of choosing an arbitrary set of gates that are then faulted simultaneously, with each gate having a limited probability of being impacted [DN23]. Consequently, when a fault is injected, a randomized subset of the selected gates becomes faulty, offering a fault adversary model much closer to physical reality. Recent work takes a significant step toward reconciling the theory of fault security proofs with physical reality by (1) introducing a more comprehensive model called the general random fault model and (2) integrating this model into formal verification tools such as VERICA [RFSG22] and IronMask [BMRT22], enabling automated security proofs [BFG⁺24]. Unlike the random fault model, which assumes the same independent fault probability for each gate, the general model accommodates an arbitrary distribution over the set of fault combinations. While this approach is more versatile, it also presents a significant challenge: verifying a design under the general random fault model requires the user to define a probability distribution tailored to the particular use case. This shifts a critical step of automated verification to the user, as no existing tool is currently capable of computing such distributions. However, it is necessary to explicitly define the set of possible fault combinations in order to construct a probability distribution over fault combinations. Since modern protected cryptographic circuits contain thousands of gates, even the manual selection of all possible combinations of adjacent gates to model a precise adversary as suggested in [BFG⁺24] is infeasible. This problem can be avoided in their model by exhaustively verifying all potential sets of fault combinations. However, such an exhaustive verification becomes computationally infeasible, as the tool must account for all possible sets of fault combinations, although some may be unrealistic or irrelevant for specific practical adversaries. Even the number of combinations in a single set poses significant challenges when assuming an adversary with limited precision, as such an adversary causes a large set of gates G^∞ to be potentially faulted. Therefore, the authors of [BFG⁺24] introduced a threshold β in their exhaustive verification procedure to limit the verification to fault combinations containing only $k \leq \beta$ faults. All fault combinations containing more than β faults assumed to be effective. Due to the exhaustiveness of their approach, they compute an accurate lower bound of the success probability of an adversary in the random fault model. However, due to β , the upper bound can be quite imprecise, depending on the number of non-evaluated fault combinations.

Table 1: High-level comparison of VERICA⁺, IronMask⁺, and FIESTA. Where (μ^ℓ, μ^u) denotes the bounds of the success probability. μ^ℓ is the lower bound, while μ^u is the upper bound.

Tool	Abstraction Level	Error Prob.	Circuit Size	Runtime	(μ^ℓ, μ^u)
IronMask ⁺	Behavioral Level	0	Small	High	Equations
VERICA ⁺	Gate Level	0	Medium	Medium	(15, 16)
FIESTA	Gate Level	$\alpha > 0$	Large	Low	Equations
	Physical Level				(14, 12)

1.1 Our Contributions

This work focuses on practical, accurate, and automated security evaluation of hardware designs that address the aforementioned challenges. The primary contribution of this work is FIESTA, an open-source fault evaluation framework that employs a statistical approach to assess circuits within the general random fault model. In particular, FIESTA is publicly available via [GitHub](#)¹ and a comparison of VERICA⁺, IronMask⁺, and FIESTA is summarized in Table 1.

Our approach is not exhaustive in terms of inputs or fault combinations, enabling the evaluation of much larger designs in less time compared to VERICA⁺ and IronMask⁺. However, since VERICA⁺ and IronMask⁺ are exhaustive in their inputs, it is guaranteed that actual success probability lies within the reported interval. In contrast, FIESTA introduces an error probability α , denoting the chance that the actual probability falls outside the bounds reported by FIESTA. Although FIESTA theoretically supports modeling local and global fault effects if the user manually defines the underlying probability distribution, it can also automatically incorporate physical information from the circuit, provided as a Graphic Design System II (GDSII) file, something that VERICA⁺ and IronMask⁺ cannot consider. This feature allows FIESTA identifying the potential set of gates that can be simultaneously faulted by a local fault injection mechanism, based on their physical placement. All features of FIESTA are demonstrated through a set of case studies, covering circuits with different fault-correction strategies and varying security levels.

2 Background

2.1 Notation

Lowercase letters represent individual elements of a group, of a finite field, or of a set, for example, $x \in \mathbb{F}_2$. Uppercase letters denote vectors or sets of distinct elements, such as $x \in X$. The cardinality of the set X is given as $|X|$. Further, we use sans-serif fonts to denote Boolean functions over n inputs such as $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$. We denote the probability of a certain event ϵ as $Pr[\epsilon]$. Considering tuples, e.g., $\psi = (g, \tau)$, we denote their elements with the identifier of the tuple in the subscript, e.g., g_ψ representing the g of ψ (resp. τ_ψ representing the τ of ψ). In general, by means of a subscript, we denote that an element belongs to another element, while we use superscripts to add further information to a specific element or set.

2.2 Physical Fault Attack Vectors

The application of Fault Injection Analysis (FIA) necessitates the induction of faults into the circuit under test, irrespective of the specific analysis technique employed. Various

¹<https://github.com/ChairImpSec/FIESTA>

physical fault injection methods are available, including clock glitching, voltage glitching, EMFI, and laser fault injection. We provide an overview of these attack vectors and categorize them based on their *local* or *global* impact.

2.2.1 Global Fault Effects

Clock glitches exploit violations of the *global* timing constraints in a synchronous circuit. These constraints can be formalized as shown in Equation (1), where T_{CLK} denotes the clock period of the circuit, $T_{D_{\text{setup}}}$ represents the time a signal at the input of a register must be stable to be sampled correctly, $T_{D_{\text{clk2Q}}}$ gives the time required for a register to update its output Q after a rising clock edge, $T_{C_{\text{max}}}$ is the maximum propagation delay of a signal from the output of one register to the input of another (through combinational logic), and $T_{D_{\text{skew}}}$ denotes the maximum phase difference between the clock signal inputs of any two registers.

$$T_{\text{CLK}} > T_{D_{\text{setup}}} + T_{D_{\text{clk2Q}}} + T_{C_{\text{max}}} + T_{D_{\text{skew}}}. \quad (1)$$

Reformulating Equation (1) allows deriving the timing slack T_{slack} of the circuit by computing:

$$T_{\text{slack}} = T_{\text{CLK}} - (T_{D_{\text{setup}}} + T_{D_{\text{clk2Q}}} + T_{C_{\text{max}}} + T_{D_{\text{skew}}}). \quad (2)$$

As long as $T_{\text{slack}} \geq 0$, Equation (1) remains satisfied, ensuring that no faults occur due to timing violations. A *clock glitch*, however, introduces a parasitic pulse into the clock tree, effectively shortening the time interval between two impulses T_{CLK} . Since all other time periods are not altered using this attack vector, Equation (2) is recalculated in the presence of a clock glitch, resulting in $T_{\text{slack}} < 0$. Consequently, Equation (1) is violated. Registers connected to data inputs that have not been fully computed within the shortened T_{CLK} interval will sample incorrect values, leading to the occurrence of faults.

Voltage glitches, on the other hand, disrupt the time required by combinational logic to compute the correct output based on the incoming inputs. Specifically, voltage glitches intentionally manipulate $T_{C_{\text{max}}}$, causing a violation of the timing slack ($T_{\text{slack}} < 0$) and resulting in a fault. This attack vector is also considered *global*, as the adversary has no direct control over the precise location of the faults. Instead, the fault location depends on the location of the register connected to the critical path of the circuit.

2.2.2 Local Fault Effects

Laser Fault Injection involves irradiating the surface of the circuit under test with a focused laser beam to induce faults. The laser induces ionization in the transistors of the physical gates, manipulating their internal states, as explained in [WA08, RSG23]. The effects of this ionization are confined to the area directly impacted by the laser spot. Due to its highly local nature, laser fault injection allows adversaries to precisely target even single bits [DBC⁺18, MDR⁺20]. This precision makes the technique particularly effective, enabling it to bypass state-of-the-art countermeasures, as demonstrated in [BBM⁺22].

EMFI is based on the principle of electromagnetic induction, first described by Faraday in 1831. An electromagnetic field induces an electromagnetic force in the wires of a circuit, generating currents within closed wire loops [DLM19, DLM21]. Experimental studies have observed a variety of effects caused by EMFI, leading to the proposal of different models. Some detailed investigations [WA08, BBM⁺22, RSG23] introduced the Sampling Fault Model and provided a detailed explanation of the operational principles of EMFI. These studies found that EMFI primarily manipulates the Power Delivery Network (PDN) of a circuit, which forms rectangular wire loops. Furthermore, when the EMFI emitter is positioned near the circuit's surface, the induced parasitic currents are largely confined to the area beneath the emitter's borders [DLM21]. The affected area can be approximated

as a rectangular frame, making the effects of EMFI *local*. Consequently, EMFI adversary can precisely target specific gates to induce faults. Additionally, [OGM17, DLM19] demonstrates that registers are predominantly influenced by EMFI. However, other works have observed different effects caused by EMFI, leading to alternative models [GYG⁺18, NDR⁺23]. These differing observations suggest that EMFI cannot be fully explained by a single operational principle applicable to all circuits under investigation.

2.3 Circuit Model

To account for faults injected into the same physical gate at different time instances (e.g., clock cycles), we utilize the (slightly adapted) structural circuit model defined in [CS21]. In this model, a structural circuit (without any combinational loop) is represented as a Directed Acyclic Graph (DAG) $B = (V, E)$, where the vertices are structural gates $v \in V$ and the edges are structural wires $e \in E$. Structural gates and wires are defined according to Definition 1 and Definition 4, respectively. In contrast to [CS21], Definition 1 does not distinguish between public and private parameters, as all parameters are potentially susceptible to faults on their underlying gates.

Definition 1 (Structural Gate). A structural gate $v = (I_v, f_v, l)$ is defined as a triplet, where I_v denotes the set of inputs of the gate, $f_v : \mathbb{F}_{|I_v|} \rightarrow \mathbb{F}_2$ defines the Boolean function evaluated by the gate, and $l : I_v \rightarrow \mathbb{N}$ is the latency of the inputs.

Each structural gate v has a single output determined by f_v . We further distinguish between combinational gates, which are (1) stateless and, therefore, have zero latency for every input as defined in Definition 2, and (2) sequential gates that have a latency of one for every input as defined in Definition 3. Note that *registers*, as discussed in Section 2.2, are modeled as sequential gates in our circuit model.

Definition 2 (Structural Combinational Gates). A structural combinational gate is a structural gate with $v = (I_v, f_v, I_v \rightarrow 0)$.

Definition 3 (Structural Sequential Gates). A structural sequential gate is a structural gate with $v = (I_v, f_v, I_v \rightarrow 1)$.

Definition 4 (Structural Wire). A structural wire is a pair $(v, (v', i))$, where v and v' are gates, and i represents an input index of v' . The wire connects the output of the source gate v to the i -th input of the destination gate v' .

The execution of a structural circuit $B = (V, E)$ over a set of clock cycles T is represented as another DAG with vertices $G = V \times T$ and edges $W = E \times T$. We denote such a *circuit execution* as $C = (G_C, W_C)$. Each *gate* $g = (v, t)$ and *wire* $w = (e, t)$ is associated with a time instance $t \in T$ while every wire $w = (e, t)$ connects gates based on their latency. The set of all structural sequential gates v_g^s of a circuit execution C is defined by $G_C^s \subseteq G_C$, while the set of structural combinational gates v_g^c is given by $G_C^c \subseteq G_C$. We further denote the input vector of a circuit execution with $X \in X^\infty$ and the output with Y , while X^∞ represents all possible input vectors.

2.4 Adversary Models

To model a gate $g \in G_C$ computing a faulty Boolean function $\tilde{f}_g$, we introduce a fault-transformation function $\tau : \mathbb{F}_2 \rightarrow \mathbb{F}_2$, which modifies the behavior of g by ensuring that the faulty function satisfies $\tilde{f}_g = \tau \circ f_g$. The most common fault transformations are as follows:

- stuck-at zero: $\tau^{\text{reset}}(x) : \mathbb{F}_2 \rightarrow \mathbb{F}_2 : x \mapsto 0$
- stuck-at one: $\tau^{\text{set}}(x) : \mathbb{F}_2 \rightarrow \mathbb{F}_2 : x \mapsto 1$

- toggle: $\tau^{\text{toggle}}(x) : \mathbb{F}_2 \rightarrow \mathbb{F}_2 : x \mapsto \bar{x}$

We consider an abstract adversary capable of faulting gates $g \in G_C$ of a circuit execution C with a fault transformation $\tau \in \mathcal{T}$ as defined in [Definition 5](#). $\mathcal{T}$ is the set of all fault transformations a certain adversary can introduce. For the remainder of this work, we assume $\mathcal{T} \subseteq \{\tau^{\text{set}}, \tau^{\text{reset}}, \tau^{\text{toggle}}\}$. Furthermore, by defining the gates which can be targeted by the vertices G_C of a circuit execution C , we allow transient faults, which may occur for only one time instance $t \in T$. Using this definition, it is still possible to model permanent faults. By choosing gates $g \in G_C$ (instead of wires) as fault targets, we follow the observation that most common physical attack vectors affect gates [RSG23].

Definition 5 (Fault). A fault is a pair $\psi = (g, \tau)$, where $\tau \in \mathcal{T}$ is a fault transformation and $g \in G_C$ is a gate within a circuit execution C . The fault causes the gate g_ψ to compute $\tilde{f}_g = \tau_\psi \circ f_g$ instead of f_g .

Based on [Definition 5](#), we call gate g *faulty* iff its evaluated Boolean function $\tilde{f}_g$ deviates from the intended function f_g , i.e., $\tilde{f}_g \neq f_g$. All faults applicable to gates $g \in G_C$ of a circuit execution C with fault transformation $\tau \in \mathcal{T}$ are denoted with $\Psi^{G_C, \mathcal{T}}$. Consequently, $\Psi^{G_C, \mathcal{T}}$ contains $|\Psi^{G_C, \mathcal{T}}| = |G_C| \cdot |\mathcal{T}|$ faults. We denote a set of faults, targeting only structural sequential gates, as defined in [Definition 3](#), as $\Psi^{G_C^s, \mathcal{T}}$. Conversely, $\Psi^{G_C^c, \mathcal{T}}$ denotes faults that target only structural combinational gates, as defined in [Definition 2](#). We do not allow injecting faults on the same gate $g \in G_C$ more than once in a clock cycle. This means that we consider only $\Psi \subseteq \Psi^{G_C, \mathcal{T}}$ with $g_\psi \neq g_{\psi'}, \forall \psi, \psi' \in \Psi^{G_C, \mathcal{T}}$ with $\psi \neq \psi'$. Therefore, we define all combination of faults targeting distinct gates $g \in G_C$ with $\Phi^{G_C, \mathcal{T}} = \{\Psi \subseteq \Psi^{G_C, \mathcal{T}}\}$. We emphasize that $\Phi^{G_C, \mathcal{T}}$ is a set of sets. The goal of the adversary is to induce a set of faults $\Psi \in \Phi^{G_C, \mathcal{T}}$ causing the output of C to become erroneous. The circuit execution faulted with Ψ is denoted as $\tilde{C}$. We refer to such a set of faults as *effective*, as defined in [Definition 6](#). Our evaluation process, detailed in [Section 3](#), and our adversary models, defined in [Section 2.4](#), rely solely on this definition of effectiveness. We do not consider whether such an effective fault is exploitable, resulting in an overly conservative model, since not all faults are exploitable in practice. This overly conservative nature of the model is prominent for the evaluation of the security of a circuit against FIA. It is used in frameworks such as FIVER [RSS⁺21] and VerFi [AWMN20], as well as in the definitions of the more recent random fault model [DN23] and general random fault model [BFG⁺24]. A more realistic approach that considers the exploitability of faults is presented by Feldtkeller *et al.* [FGS23]. However, this approach is computationally infeasible for complex designs. Another more realistic approach is presented in [SAB⁺23], which introduces a deep learning-based method for statistical fault-leakage detection.

Definition 6 (Effectiveness). Let $\Psi \in \Phi^{G_C, \mathcal{T}}$ represent a set of faults that target distinct gates. The fault set Ψ is *effective* with respect to (w.r.t.) the primary input vector $X \in X^\infty$, if the resulting primary output vector $\tilde{Y}$ of $\tilde{C}$ deviates from the intended primary output vector Y of C , i.e., $\tilde{Y} \neq Y$. Conversely, Ψ is *ineffective* w.r.t. X , if $\tilde{Y} = Y$.

Note that other definitions of effectiveness exist in literature. For example, in [BFG⁺24], a fault is considered effective if it results in a primary output vector $\tilde{Y}$ that differs from the intended output vector Y . According to this definition, a fault is deemed effective if it causes a deviation in the output for even a single input vector $X \in X^\infty$. Therefore, a fault is considered *ineffective* only if it holds that $Y = \tilde{Y}, \forall X \in X^\infty$. In contrast, [Definition 6](#) considers a fault effective only w.r.t. a specific input vector. The adversary has no control over the secret input elements $K \subseteq X$ of an input vector, but can freely choose the remaining elements $P \subseteq X$, where $K \cap P = \emptyset$. Each secret input bit $k \in K$ is

chosen uniformly at random, denoted as $k \sim \text{Uniform}(0, 1)^2$. The resulting distribution of $K \in K^\infty$ is denoted by $\mathbb{D}K^\infty$. In contrast, the adversary has full control over the public input bits $p \in P$ and can select the distribution $\mathbb{D}P^\infty$. This includes the ability to fix the public input vector P to a specific value. The resulting distribution over X^∞ is then given by

$$\mathbb{D}X^\infty = \mathbb{D}P^\infty \times \mathbb{D}K^\infty, \quad \text{where } X = P \cup K \text{ and } P \cap K = \emptyset. \quad (3)$$

Based on the given definitions, we formalize the threshold fault model as well as the (general) random fault model by means of security games.

2.4.1 Threshold Fault Model

We define the Threshold Fault Model following the approach suggested in [RSG23]. It offers great flexibility by imposing additional restrictions on the adversary's capabilities, tailored to the specific security goals. In addition to k , it allows specifying the set of fault transformations, which can occur. It also enables limiting the set of gates that can be targeted by a certain adversary to any subset of G_C . Note that the technique presented in [RSG23] does not limit the fault transformation to a function manipulating the output, but allows arbitrary fault transformations. However, we restrict the adversary to $\tau \in \{\tau^{\text{set}}, \tau^{\text{reset}}, \tau^{\text{toggle}}\}$, since for each circuit execution any arbitrary fault transformation can be reduced to one of the elements in τ . In general, let $\mathcal{A}^{\Psi, X}$ be a threshold fault adversary capable of simultaneously faulting up to k distinct gates of a circuit execution C . Based on this restriction, let $\Psi \in \Phi^{G_C, \mathcal{T}}$ be a set of faults with $|\Psi| = k$ selected by $\mathcal{A}^\Psi$. In contrast to our definition given above, the adversary has full control over the input X and can select it as advantageously as possible. The resulting security game is as follows.

Setup $\mathcal{A}^{\Psi, X}$ queries an oracle $\mathcal{O}$ implementing C , providing $\mathcal{O}$ with Ψ and the selected input vector X .

Simulation $\mathcal{O}$ evaluates X on a faulty circuit $\tilde{C}$, a modified version of C derived based on the k faults $\psi \in \Psi$.

Fault Diagnosis $\mathcal{O}$ returns 0 if Ψ was ineffective (i.e., the output of $\tilde{C}$ matches the expected output of C) and 1 if Ψ was effective (i.e., the outputs differ).

The advantage of $\mathcal{A}^{\Psi, X}$ is defined as $\text{Adv}(\mathcal{A}^{\Psi, X}) = \Pr[\mathcal{O}(\Psi, X) = 1] \in \{0, 1\}$. As the output of $\mathcal{O}$ is deterministic, a single query is enough to estimate $\text{Adv}(\mathcal{A}^{\Psi, X})$. $\mathcal{A}^{\Psi, X}$ is selected from the set of all possible adversaries $\mathcal{A}^\infty$. Finally, we define C as threshold k -fault secure, if $\text{Adv}(\mathcal{A}^{\Psi, X}) = 0$ for all possible $\mathcal{A}^{\Psi, X} \in \mathcal{A}^\infty$.

2.4.2 Random Fault Model

Let $\mathcal{A}^\zeta$ denote a random fault adversary with $\zeta = (\varphi, \Psi^\infty, \mathbb{D}P^\infty)$ capable of targeting a circuit execution C with an arbitrary combination of distinct faults $\Psi^\infty \in \Phi^{G_C, \mathcal{T}}$. In particular, Ψ^∞ is not limited to a specific number of elements. Each attempted fault $\psi \in \Psi^\infty$ is independently induced with probability φ . In this work, we consider the conservative correctness game [DN23] to define the φ -random fault security game as follows.

Setup $\mathcal{A}^\zeta$ queries an oracle $\mathcal{O}$ implementing C , providing $\mathcal{O}$ with Ψ^∞ and a distribution of the public inputs $\mathbb{D}P^\infty$. For every attempted fault $(g, \tau) = \psi \in \Psi^\infty$, it holds that $\tilde{f}_g = \tau \circ f_g$ with $\tau \in \mathcal{T}$.

Sampling For every $\psi \in \Psi^\infty$, $\mathcal{O}$ adds $\psi \in \Psi^\infty$ with a probability of φ to the set of faults Ψ , which are actually injected with $\Psi \subseteq \Psi^\infty$. Additionally, $\mathcal{O}$ samples X from X^∞ according to $\mathbb{D}X^\infty$.

² $k \sim \text{Uniform}(n, m)$ describes the sampling of a natural number k between n and m from a uniform distribution.

Simulation $\mathcal{O}$ evaluates X on the golden circuit C and on the faulty circuit $\tilde{C}$, a modified version of C derived according to Ψ .

Fault Diagnosis $\mathcal{O}$ returns 0 if Ψ was ineffective (i.e., the output of $\tilde{C}$ matches the output of C) and 1 if Ψ was effective (i.e., the outputs differ).

The advantage of $\mathcal{A}^\zeta$ is again defined as $Adv(\mathcal{A}^\zeta) = Pr[\mathcal{O}(\varphi, \Psi^\infty, \mathbb{D}P^\infty) = 1]$. In the random fault model, the most powerful adversary $\mathcal{A}^{\max}$ is always assumed. This means that the adversary chooses Ψ^∞ and $\mathbb{D}P^\infty$ such that it maximizes the advantage. This adversary is selected from the set of all possible adversaries $\mathcal{A}^\infty$, with an advantage given by $Adv(\mathcal{A}^{\max}) = \max_{\mathcal{A}^\zeta \in \mathcal{A}^\infty} Adv(\mathcal{A}^\zeta)$. We assume φ -random fault security if $Adv(\mathcal{A}^{\max})$ is bounded by some (acceptable small) μ , i.e., $Adv(\mathcal{A}^{\max}) \leq \mu$.

2.4.3 General Random Fault Model

A recent paper [BFG⁺24] introduced a generalization of the random fault adversary, referred to as the *general random fault model*. The key differences from the random probing model are outlined below.

- The general random fault model is not restricted to a single independent probability for all faults. Instead, it allows for arbitrary probability distributions, including joined probabilities for multiple faults. Consequently, an adversary in the general random fault model has to be defined by a set of fault combinations $\Phi^\infty \subseteq \Phi^{G_C, \mathcal{T}}$ and a probability distribution $\mathbb{D}\Phi^\infty$, which defines the distribution of the fault combinations. To represent a random fault adversary $\mathcal{A}^{\zeta'}$ with $\zeta' = (\varphi, \Psi^\infty, \mathbb{D}P^\infty)$ using a general random fault adversary $\mathcal{A}^\zeta$ with $\zeta = (\mathbb{D}\Phi^\infty, \Phi^\infty, \mathbb{D}P^\infty)$, the following observation applies: In the random fault model each fault $\psi \in \Psi^\infty$ is sampled independently with probability φ . The set of all possible fault combinations can be expressed in terms of the general random fault model as $\Phi^\infty = \{\Psi \subseteq \Psi^\infty\}$. Each combination $\Psi \in \Phi^\infty$ contains $|\Psi|$ active faults, each of which is sampled with probability φ , while the remaining $(|\Psi^\infty| - |\Psi|)$ faults are inactive (not part of Ψ). The probability that a fault is not part of the set Ψ is given by $(1 - \varphi)$. Since faults in the random fault model are sampled independently, the probability of a specific combination $\Psi \in \Phi^\infty$ occurring is given as follows.

$$\mathbb{D}\Phi^\infty[\Psi] = \varphi^{|\Psi|} \cdot (1 - \varphi)^{(|\Psi^\infty| - |\Psi|)} \quad (4)$$

- The general random fault model permits arbitrary fault transformation functions $\mathbb{F}_{2^n} \rightarrow \mathbb{F}_2$, analogue to [RSG23], in contrast to the restricted set $\{\tau^{\text{set}}, \tau^{\text{reset}}, \tau^{\text{toggle}}\}$ we, and the random fault model, consider. As stated, considering Definition 6, any of the fault transformation functions considered in [BFG⁺24] can be mapped to τ^{set} , τ^{reset} , or τ^{toggle} for each circuit execution.
- The scope of this model is restricted to a single adversary $\mathcal{A}^\zeta$, which is not necessarily the strongest adversary $\mathcal{A}^{\max}$ of the random fault model.
- In this model, a fault is assumed to be effective if there exists an input vector X of C for which the primary output vector $\tilde{Y}$ deviates from the intended primary output vector Y . In contrast, the random fault model considers a fault to be effective w.r.t. an input vector X , as defined in Definition 6. We emphasize that our definition of the general random fault model employs Definition 6 and, therefore, differs from the definition given in [BFG⁺24]. This deviation is motivated by the need to assess large circuits under the general random fault model, which would otherwise be computationally infeasible.

Now, let $\mathcal{A}^\zeta$ be a general random fault adversary with $\zeta = (\mathbb{D}\Phi^\infty, \Phi^\infty, \mathbb{D}P^\infty)$ capable of inducing one fault set $\Psi \in \Phi^\infty$ in a circuit execution C . The adversary has no control

over which fault set Ψ is induced, but only knows that it is one of the fault combinations given by Φ^∞ . The probability that each $\Psi \in \Phi^\infty$ occurs is defined by the probability distribution $\mathbb{D}\Phi^\infty$. The authors of [BFG⁺24] suggest choosing Φ^∞ to reflect all structural sequential gate combinations with long critical paths (in order to model clock glitches), or to construct it to holds for all combinations of adjacent gates (in order to model laser attacks). The general random fault security game is then defined as follows.

Setup $\mathcal{A}^\zeta$ queries an oracle $\mathcal{O}$ implementing C , providing $\mathcal{O}$ with Φ^∞ and a distribution of public input elements $\mathbb{D}P^\infty$.

Sampling $\mathcal{O}$ samples Ψ randomly from Φ^∞ according to $\mathbb{D}\Phi^\infty$, and X randomly from X^∞ according to $\mathbb{D}X^\infty$.

Simulation $\mathcal{O}$ evaluates X on the golden circuit C and the faulty circuit $\tilde{C}$, a modified version of C derived from Ψ .

Fault Diagnosis $\mathcal{O}$ returns 0 if Ψ was ineffective and 1 if Ψ was effective.

The advantage of $\mathcal{A}^\zeta$ is defined as $Adv(\mathcal{A}^\zeta) = Pr[\mathcal{O}(\mathbb{D}\Phi^\infty, \Phi^\infty, \mathbb{D}P^\infty) = 1]$. We assume $\mathbb{D}\Phi^\infty$ -random fault security, if $Adv(\mathcal{A}^\zeta)$ is bounded by some (acceptable small) μ , i.e., $Adv(\mathcal{A}^\zeta) \leq \mu$.

2.5 Countermeasures

In this work, we focus on error-correction techniques as a countermeasure against FIA. These techniques generally introduce information redundancy in time and/or spatial domain. The redundant operations are complemented by an error-correction module, which can employ methods such as simple majority voting or advanced error-correcting codes. It can be formally shown that an error-correction module is capable of correcting up to t faults, thereby guaranteeing t -threshold security when applied at every clock cycle. However, the associated overhead (in time and/or spatial domain) increases with t , making it common for designs to be protected against only a few faults.

3 Oracle Inspired Verification

Our automated verification approach is inspired by the security games in Section 2.4.2 and Section 2.4.3. Specifically, we repeat the *Sampling*, *Simulation*, and *Fault Diagnosis* steps of the (general) random fault security game n times and count the number of effective faults n^{eff} . Based on n^{eff} , we statistically estimate a confidence interval $[\mu^\ell, \mu^u]$ for the advantage $\mu = Adv(\mathcal{A}^\zeta)$ of an adversary $\mathcal{A}^\zeta$ targeting C .

3.1 Sampling

To assess the security of a circuit execution C against an adversary $\mathcal{A}^\zeta$ using our oracle-inspired approach, we sample a set of faults $\Psi \in \Phi^\infty$ injected into C and an input $X \in X^\infty$. The samples are generated based on the probability distributions characterizing $\mathcal{A}^\zeta$ as detailed in Section 2.4.2 and Section 2.4.3. We use a Mersenne Twister (MT) algorithm with a period length of $2^{19937} - 1$ [MN98] as our Pseudo-Random Number Generator (PRNG). This algorithm serves as the random source for all sampling processes. Fault sampling and input sampling utilize separate internal states of the MT algorithm.

3.1.1 Fault Sampling (FS)

We maintain separate sampling processes for random fault adversaries and general random fault adversaries, despite the possibility of unifying them, as discussed in Section 2.4. This separation simplifies the configuration and enables performance optimizations leveraging

the unique characteristics of the random fault model. Consequently, we implement two distinct sampling strategies, each tailored to one of the two fault models. Both strategies can model non-uniform behavior in fault occurrences, as the fault combinations $\Psi \in \Phi^\infty$ are not necessarily uniformly distributed, even if each individual fault occurs with the same probability φ . A uniform distribution over fault combinations only arises if $\varphi = \frac{1}{2}$, assuming equal likelihood for each fault. The strategy outlined in Section 3.1.1 supports all distributions where each individual fault occurs with a certain probability φ , allowing the user to specify φ directly. In contrast, the strategy in Section 3.1.1 allows the specification of an individual probability for each fault combination, enabling the modeling of distributions that cannot be described by a single φ . While we impose no restrictions on the fault distributions, we limit the adversary to fault transformations $\mathcal{T} \subseteq \{\tau^{\text{set}}, \tau^{\text{reset}}, \tau^{\text{toggle}}\}$, and only allow $\Phi^\infty \subseteq \Phi^{G_C, \mathcal{T}}$. This flexibility enables FIESTA to model any physical effect represented by an arbitrary probability distribution and the fault transformations $\tau \in \mathcal{T}$. Additionally, FIESTA allows the user to exclude specific faults from the security assessment. For example, in the context of DFA, only faults injected during specific rounds of the AES are typically considered exploitable, as they contribute to an efficient and exploitable distinguisher. In such cases, the user can exclude faults from the other rounds, as they are considered non-exploitable, even when effective. While modeling the effects through probabilities remains the user's responsibility, we provide an extension to assist in selecting possible fault combinations under certain physical conditions, as detailed in Section 6.2.

φ -Fault Sampling (φ -FS). A random fault adversary $\mathcal{A}^\zeta$, as defined in Section 2.4.2, is characterized by the tuple $\zeta = (\varphi, \Psi^\infty, \mathbb{D}P^\infty)$ consisting of a set of distinct faults Ψ^∞ , a probability φ , and a selected distribution of the public inputs $\mathbb{D}P^\infty$. While all three parameters are configurable by the user, enabling customization of adversaries to meet specific requirements, only the first two elements of the tuple impact the φ -fault sampling. Note, that φ and Ψ^∞ must be provided in any case, either fully specified by the user or derived using the extension detailed in Section 6.2. A random fault adversary attempts to inject each fault $\psi \in \Psi^\infty$, where each fault is equally likely to be injected with probability φ . Therefore, we model the fault injection using a Bernoulli distribution, $\text{Bernoulli}(\varphi)$, which enables sampling each fault $\psi \in \Psi^\infty$ with probability φ . Internally, $\text{Bernoulli}(\varphi)$ samples a uniform real number t with $0 \leq t < 1$ and compares it with φ . If $t \leq \varphi$, $\text{Bernoulli}(\varphi)$ returns true. If a fault is sampled, it is inserted into another set Ψ , which includes all faults considered active, i.e., those that will actually manifest in the circuit. The procedure is illustrated in Algorithm 1.

Algorithm 1 φ -Fault Sampling (φ -FS).

```

1: Input:  $\Psi^\infty, \varphi$ 
2: Output:  $\Psi$ 
3:  $\psi \leftarrow \emptyset$ 
4: for all  $\psi \in \Psi^\infty$  do                                 $\triangleright$  Iterate through all faults specified by the user
5:   if  $\text{Bernoulli}(\varphi) = \text{true}$  then                         $\triangleright$  Returns true with probability  $\varphi$ 
6:      $\Psi \leftarrow \Psi \cup \{\psi\}$                          $\triangleright$  Add the fault  $\psi$  to the set of active faults  $\Psi$ 
7: return  $\Psi$ 

```

$\mathbb{D}\Phi^\infty$ -Fault Sampling ($\mathbb{D}\Phi^\infty$ -FS). According to Section 2.4.3, a general random fault adversary $\mathcal{A}^\zeta$ is characterized by the tuple $\zeta = (\mathbb{D}\Phi^\infty, \Phi^\infty, \mathbb{D}P^\infty)$. Therefore, in contrast to a random fault adversary this tuple contains a probability distribution $\mathbb{D}\Phi^\infty$ over the set of all possible fault combinations Φ^∞ , instead of a single probability φ with that each

independent fault targeted by the adversary $\psi \in \Psi^\infty$ occurs. To facilitate the customization of such a general adversary, we allow the user to define $\mathcal{A}^\zeta$ in the following ways.

- If every fault should occur with an *independent* probability, the user can specify a set of tuples Ψ' with $(\psi, \varphi_\psi) \in \Psi'$ while every tuple consists of a single fault ψ together with its associated probability φ_ψ . In this case, the sampling can be performed using a slightly modified version of [Algorithm 1](#), which now samples based on individual probabilities for each fault rather than a single probability for all faults.
- If the probabilities of the individual faults are *dependent*, the user can directly define Φ^∞ and $\mathbb{D}\Phi^\infty$ by assigning an individual probability to each fault set $\Psi \in \Phi^\infty$. In this scenario, sampling cannot be performed using a Bernoulli distribution. Instead, a more sophisticated approach, such as the weighted random sampling algorithm presented in [Algorithm 2](#), is required. We note that Lines 5-7 describe the process of generating a weight for each $\Psi \in \Phi^\infty$. This serves as a pre-processing step and needs to be executed only once. The actual sampling is performed in Line 8, where a random number is uniformly sampled between 0 and 1. This number is then compared against the set of weights W in Lines 9-12. If the randomly sampled number p satisfies $w_{i-1} < p \leq w_i$ where $w_{i-1}, w_i \in W$, then the fault set Ψ_i is selected. This sampling process ensures that each Ψ_i is sampled with $Pr_{\mathbb{D}\Phi^\infty}[\Psi_i]$.

Algorithm 2 $\mathbb{D}\Phi^\infty$ -Fault Sampling ($\mathbb{D}\Phi^\infty$ -FS).

```

1: Input:  $\Phi^\infty, \mathbb{D}\Phi^\infty$ 
2: Output:  $\Psi$ 
3:  $p \leftarrow 0$ 
4:  $W \leftarrow \emptyset$ 
▷ Pre-processing

5: for all  $\Phi \in \Phi^\infty$  do
6:    $p \leftarrow p + Pr_{\mathbb{D}\Phi^\infty}[\Psi]$ 
7:    $W \leftarrow W \cup \{p\}$ 
▷ Sampling

8:  $p \xleftarrow{\$} \text{Uniform}(0, 1)$  ▷ Uniformly sample a random real number between zero and one
9: for  $i = 0, \dots, |W| - 1$  do
10:   if  $p \leq w_i \in W$  then
11:      $\Psi \leftarrow \Psi_i \in \Phi^\infty$ 
12:   return  $\Psi$ 

```

3.1.2 Input Sampling (IS)

The second sampling process, performed by an oracle $\mathcal{O}$, samples the input $X \in X^\infty$ according to $\mathbb{D}X^\infty$. While the distribution $\mathcal{D}P^\infty$ over the public input elements can be chosen arbitrary by the adversary according to our adversary model, the private input elements are always sampled uniformly at random, as explained in [Section 2.4](#). Our implementation imposes a constraint on the adversaries that can be evaluated, as the distribution of public inputs $\mathcal{D}P^\infty$ is limited to combinations of fixed and uniformly sampled bits. This differs from the sampling step of oracles in random fault models, which allows the adversary to define any distribution over possible public input combinations P^∞ . This limitation arises from the basis of our implementation, as detailed in [Section 3.2](#). While theoretically restrictive, it has minimal impact on realistic adversaries who either select constant input vectors based on known circuit behavior or sample inputs uniformly at random when specific knowledge of the circuit is unavailable.

3.2 Simulator (Sim)

Several simulators capable of simulating Verilog gate-level netlists have been introduced through academic research or are (commercially) available. Notable examples from academia include the simulators developed for VerFi [AWMN20] and PROLEAD [MM22], and, more recently, Quantile [HCP⁺24]. Additionally, Verilator³ is widely recognized and publicly available. For this work, we choose the simulator of VerFi and PROLEAD. Notably, VerFi is designed for fault diagnosis, and its built-in support for fault injection aligns well with our requirements. However, Quantile’s simulator offers greater efficiency in terms of simulation time compared to the one developed for VerFi. We believe integrating Quantile’s simulator could further enhance the efficiency of our tool in future iterations. The simulator requires a gate-level netlist written in Verilog, accompanied by a cell library that defines all logical gates instantiated in the netlist. Specifically, the cell library specifies the Boolean function f_g computed by each gate in the netlist. The netlist is parsed into a structural circuit, which is then simulated over a predefined set of cycles T , following the approach described in Section 2.3. During the simulation, which is initialized by a predefined input sequence, the signals transmitted through each wire during the circuit execution are computed using a time-driven approach. To accelerate the simulation process, each signal is represented as a 64-bit variable, enabling the simultaneous handling of 64 independent simulations in parallel through bit-slicing. Furthermore, we note that additional blocks of 64 simulations can be handled in parallel through the use of multi-threading. Hence, the simulator returns the final simulation as a set of n_t 64-bit variables S effectively storing $64n_t$ independent results for each wire in the circuit execution.

To evaluate the effectiveness of a set of injected faults Ψ , the simulated outputs of two circuit executions are compared using the same input sequence. The first simulation is conducted without faults, i.e., $\Psi = \emptyset$, resulting in what is referred to as a golden state S . In the second simulation, each fault $\psi \in \Psi$ is injected, resulting in a modified state $\tilde{S}$. Simulating the golden state is straightforward; however, effectively injecting faults during the simulation requires the creation of a mapping function, $MG_C \rightarrow \mathcal{T}$. This function maps each gate $g \in G_C$ in the circuit execution to either $\tau^{id} : \mathbb{F}_2 \rightarrow \mathbb{F}_2 : x \mapsto x$, if the gate is not affected by a fault, or to its corresponding fault-transformation function $\tau \in \{\tau^{\text{set}}, \tau^{\text{reset}}, \tau^{\text{toggle}}\}$. Thus, when simulating a single $g \in G_C$, the simulator computes $\tilde{f}_g = \tau_g \circ f_g$ while $\tau_g = M(g)$. This approach minimizes the overhead of fault injection to the computation of τ_g and a single table lookup per gate.

3.3 Fault Diagnosis (FD)

To evaluate the effectiveness of the fault set Ψ w.r.t. the input X , the outputs Y and $\tilde{Y}$ – derived from X – must be compared. According to Section 3.2, the simulator computes the golden and faulty states, S and $\tilde{S}$, of the circuit given the input X and the fault set Ψ . The outputs Y and $\tilde{Y}$ are then extracted from the respective states S and $\tilde{S}$. The comparison is repeated for all simulations and the count of effective faults n^{eff} is calculated.

3.4 Statistical Analysis (SA)

The approximation of $Adv(\mathcal{A}^\zeta)$ for a given adversary $\mathcal{A}^\zeta$ against a circuit C by a confidence interval serves as the foundation for our implementation of FIESTA. We consider a general random fault adversary $\mathcal{A}^\zeta$, where $\zeta = (\mathbb{D}\Phi^\infty, \Phi^\infty, \mathbb{D}P^\infty)$. This imposes no restriction on our approach, as any random fault adversary can be represented by a general random fault adversary, as shown in Section 2.4.3. $Adv(\mathcal{A}^\zeta)$ is defined as the probability that the oracle $\mathcal{O}$ outputs 1, where $\mu = Pr[\mathcal{O}(\zeta) = 1]$.

³<https://www.veripool.org/verilator/>

The oracle $\mathcal{O}$ outputs 1, iff the sampled fault Ψ is effective w.r.t. the sampled X ; otherwise it outputs 0. Therefore, $\mathcal{O}(\zeta)$ is a Bernoulli random variable with success probability μ . The oracle $\mathcal{O}(\zeta)$ depends on the circuit C , the distribution of the inputs $\mathbb{D}X^\infty$, and the distinction of the fault combinations $\mathbb{D}\Phi^\infty$. Based on this observation, the success (a fault is effective) probability can be computed as

$$\mu = \sum_{\Psi \in \Phi^\infty} \sum_{X \in X^\infty} Pr_{\mathbb{D}\Phi^\infty}[\Psi] \cdot Pr_{\mathbb{D}X^\infty}[X] \cdot E(C, \Psi, X), \quad (5)$$

where $E(C, \Psi, X)$ is defined as

$$E(C, \Psi, X) = \begin{cases} 1, & \text{if } \Psi \text{ is effective w.r.t. } X \text{ for circuit } C \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

To compute μ accurately, as described in [BFG⁺24], the effectiveness of all combinations $(\Psi, X) \in \Phi^\infty \times X^\infty$ must be evaluated. While this approach is feasible for small sets Φ^∞ and X^∞ , as demonstrated in [BFG⁺24], it becomes impractical when analyzing a precise adversary against a complex cryptographic circuit. Exhaustively testing all possible secret inputs for such a circuit is computationally prohibitive. The challenge is further exacerbated for adversaries with low spatial resolution, as the number of potential fault combinations grows exponentially. In particular, the size of Φ^∞ is calculated by

$$|\Phi^\infty| = \sum_{k=1}^{|G^\infty|} \binom{|G^\infty|}{k} \cdot |\mathcal{T}|, \quad (7)$$

where all combinations of gates $g \in G^\infty$ are assumed to be faulted. According to the binomial theorem, Equation (7) can be simplified to $|\Phi^\infty| = (2^{|G^\infty|} - 1) \cdot |\mathcal{T}|$. This simplification illustrates that the complexity of a naive verification procedure, which exhaustively computes μ , increases exponentially with both the number of possible input combinations for the secret *and* the number of gates targeted by the adversary. Consequently, a naive exhaustive verification strategy is restricted to precise adversaries and circuits with only a limited number of secret inputs. One objective of the random fault model is to avoid constraining adversaries to fault only small sets of gates simultaneously. To address the computational limitations of the model, we propose a statistical method for approximating μ at a specified confidence level γ . This approach allows the verification of large and complex circuits against arbitrary adversaries. To approximate μ , we leverage the observation that N^{eff} , representing the number of successes (i.e., count of $\mathcal{O}(\zeta) = 1$) out of n independent and identical evaluations of $\mathcal{O}(\zeta)$ (trials), follows a Binomial distribution: $N^{\text{eff}} \sim \text{Binomial}(n, \mu)$. Using the Cumulative Density Function (CDF) of the Binomial distribution, the probability of observing at most n^{eff} successes is given by

$$Pr_{n,\mu}[N^{\text{eff}} \leq n^{\text{eff}}] = \sum_{i=0}^{n^{\text{eff}}} \binom{n}{i} \cdot \mu^i \cdot (1 - \mu)^{n-i}. \quad (8)$$

To estimate μ , we solve Equation (8) for a given confidence level γ , based on which the corresponding miss probability is $\alpha = 1 - \gamma$. We determine an upper bound μ^u and a lower bound μ^ℓ for μ at the confidence level γ . Therefore, the miss probability for one boundary is given by $\frac{\alpha}{2}$. To derive the upper bound μ^u , we have to solve Equation (8) under the following condition:

$$Pr_{n,\mu}[N^{\text{eff}} \leq n^{\text{eff}}] = \frac{\alpha}{2}. \quad (9)$$

Since $Pr_{n,\mu}[N^{\text{eff}} \leq n^{\text{eff}}]$ decreases as μ increases, solving this equation provides the largest μ ensuring that at most n^{eff} successes are observed with probability $\frac{\alpha}{2}$. For $\mu > \mu^u$ the

probability $Pr_{n,\mu}[N^{\text{eff}} \leq n^{\text{eff}}]$ becomes smaller than $\frac{\alpha}{2}$, meaning that observing n^{eff} or fewer successes occurs with less than the specified confidence level. Thus, μ^u represents the largest value of μ for which it can be assumed that n^{eff} or fewer successes are observed for confidence level γ . Instead of directly solving Equation (9), we exploit that the CDF of the Binomial distribution can be expressed by the regularized incomplete beta function:

$$Pr_{n,\mu}[N^{\text{eff}} \leq n^{\text{eff}}] = 1 - I_\mu(n^{\text{eff}} + 1, n - n^{\text{eff}}). \quad (10)$$

For a proof of Equation (10) and additional details on the deviation of the confidence interval, we refer to [Sch08]. The solution of

$$I_\mu(n^{\text{eff}} + 1, n - n^{\text{eff}}) = 1 - \frac{\alpha}{2} \quad (11)$$

solved for μ can be derived efficiently using numerical methods, such as Newton's method and Halley's method. We denote the computation of this solution using the inverse regularized incomplete beta function with

$$\mu^u = I^{-1}\left(1 - \frac{\alpha}{2}, n^{\text{eff}} + 1, n - n^{\text{eff}}\right). \quad (12)$$

We refer to the implementation⁴ provided by the C++ Boost framework [Boo15] for a realization of an algorithm to numerically solve Equation (12).

The lower bound μ^ℓ , on the other hand, is determined as the smallest value of μ such that n^{eff} or more successes are obtained with probability $\frac{\alpha}{2}$. This is equivalent to solve Equation (13) for μ :

$$Pr_{n,\mu}[N^{\text{eff}} \geq n^{\text{eff}}] = 1 - Pr_{n,\mu}[N^{\text{eff}} \leq (n^{\text{eff}} - 1)] = \frac{\alpha}{2}. \quad (13)$$

This can be solved efficiently too, by exploiting the aforementioned connection between the Beta distribution and the Binomial distribution. Analogue to Equation (12) we denote the computation of the solution with

$$\mu^\ell = I^{-1}\left(\frac{\alpha}{2}, n^{\text{eff}}, n - n^{\text{eff}} + 1\right). \quad (14)$$

In the literature, the resulting interval $[\mu^\ell, \mu^u]$ is referred to as the Clopper-Pearson Interval. It provides a conservative estimation of the confidence boundaries, as it guarantees that the coverage probability (the probability with which $\mu^\ell \leq \mu \leq \mu^u$ holds for the given parameters) is at least γ [Sch08]. This conservative nature arises because the interval accounts for the possibility that the actual coverage probability may exceed γ . For security assessments, this is a desirable property, as it ensures that even in the worst case, the guarantee $\mu \in [\mu^\ell, \mu^u]$ holds.

3.5 Gate-Level Evaluation Using FIESTA

Using the components explained above, FIESTA enables the verification under both the random fault model and the general random fault model, as defined in Section 2.4. In the random fault model, verification involves evaluating the circuit against all possible adversaries $\mathcal{A}^\infty$ to identify the most powerful $\mathcal{A}^{\text{max}}$. In contrast, the general random fault model narrows its focus to verify a circuit C against one specified adversary $\mathcal{A}^\zeta$. In the following, we explain our verification procedure for a single adversary $\mathcal{A}^\zeta$. From this foundation, only minor extensions are required to enable verification of a set of adversaries. Figure 1 provides an overview of the flow of operations required to assess a circuit C against a certain adversary $\mathcal{A}^\zeta$.

⁴https://www.boost.org/doc/libs/1_87_0/libs/math/doc/html/math_toolkit/sf_beta/ibeta_inv_function.html

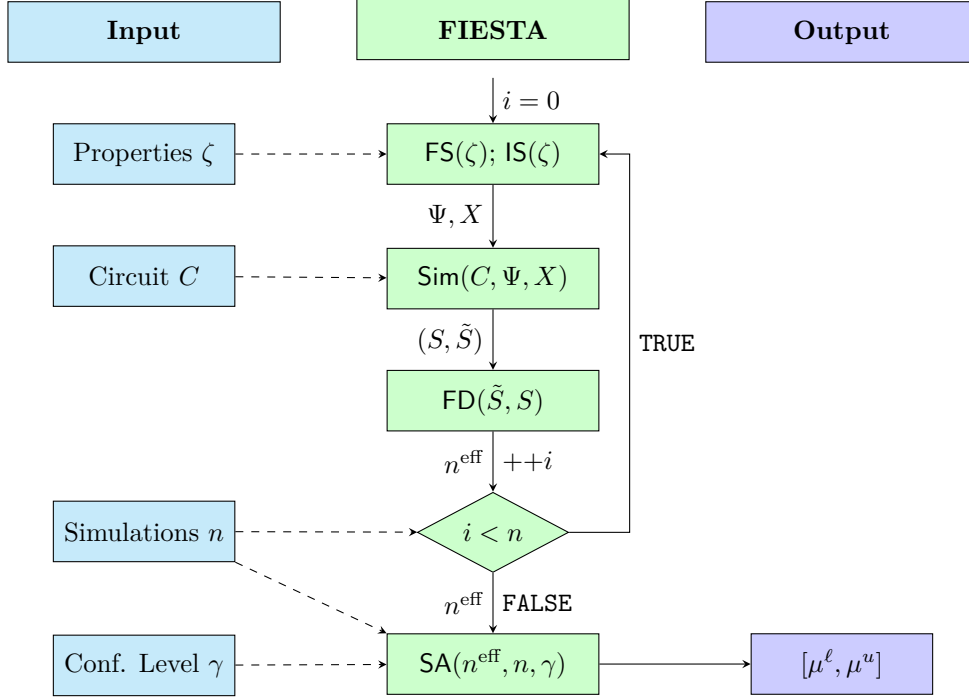


Figure 1: Flow of FIESTA to evaluate a circuit against a single adversary $\mathcal{A}^\zeta$. The right side of the arrows shows data relevant to the control flow, while the left side visualizes the data flow through FIESTA.

We emphasize that ζ can be selected to either defining a random fault adversary with $\zeta = (\varphi, \Psi^\infty, \mathbb{D}P^\infty)$ or a general random fault adversary with $\zeta = (\mathbb{D}\Phi^\infty, \Phi^\infty, \mathbb{D}P^\infty)$. As depicted in Figure 1, FIESTA requires a circuit C and the properties ζ defining $\mathcal{A}^\zeta$ as input. The properties ζ are defined in a configuration file passed to FIESTA together with the Verilog gate-level netlist of C . These settings are then parsed, and the appropriate fault-sampling method – either the φ -Fault Sampling or the $\mathbb{D}\Phi^\infty$ -Fault Sampling – is selected based on the chosen adversary type, as explained for φ -FS and $\mathbb{D}\Phi^\infty$ -FS, respectively (cf. Section 3.1.1). Using the selected fault sampling method, denoted by FS in Figure 1, a fault Ψ is sampled. The input is sampled by IS as explained in Section 3.1.2. Based on the sampled input vector $X \in X^\infty$ and the sampled fault combination $\Psi \in \Phi^\infty$, the tuple $(S, \tilde{S})$ of the circuit execution C is simulated through the simulator $\text{Sim}(C, \Psi, X)$, as detailed in Section 3.2. By extracting and comparing Y and $\tilde{Y}$ from $(S, \tilde{S})$ the effectiveness of the corresponding fault is derived by the fault diagnosis function $\text{FD}(S, \tilde{S})$, as explained in Section 3.3. During this process, n^{eff} is naturally incremented for each effective fault.

The simulator leverages vectorization and multi-threading to compute $64 \times n_t$ independent results simultaneously. Consequently, the selected sampling process is repeated 64 times on each of the n_t threads to provide the simulator with the required number of independently sampled fault combinations, i.e., $64 \times n_t$. These independent samples are required due to the used statistical analysis explained in Section 3.4.

After all n simulations are performed and n^{eff} faults are observed, FIESTA computes the Clopper-Pearson Interval $[\mu^\ell, \mu^u]$, as described in Section 3.4, using the statistical analysis function $\text{SA}(n^{\text{eff}}, n, \gamma)$ for the confidence level γ defined in the configuration file. The computed interval $[\mu^\ell, \mu^u]$ bounds the advantage $\text{Adv}(\mathcal{A}^\zeta)$ of one specific adversary. To determine the strongest adversary $\mathcal{A}^{\max}$, FIESTA supports the evaluation of $n^{\mathcal{A}}$ adversaries. Each adversary $\mathcal{A}^{\zeta_i}$ with $i \in \{0, \dots, n^{\mathcal{A}} - 1\}$, must be specified in the configuration file.

The tool computes the confidence interval $[\mu^\ell, \mu^u]$ for each adversary and reports the adversary with the largest upper bound μ^ℓ as $\mathcal{A}^{\max}$.

3.6 An Illustrative Example

To clarify the steps described above, we provide a manual analysis of an example circuit C , as illustrated in Figure 2, against an adversary $\mathcal{A}^\zeta$. As C with only three gates

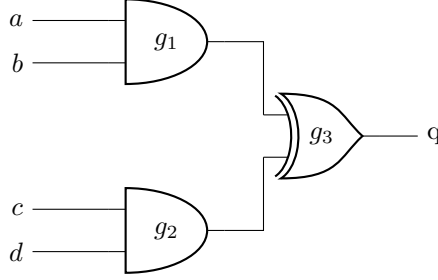


Figure 2: Example of a simple low-complexity circuit execution C tailored for manual and exhaustive analysis.

$G_C = \{g_1, g_2, g_3\}$ is very small, we can compute the exact $\mu = Adv(\mathcal{A}^\zeta)$ by exhaustively analyzing every input and attack vector. Assume that the circuit receives four secret input signals, so we have $P^\infty = \emptyset$ and $X^\infty = \mathbb{F}_2^4$ resulting in a set of 16 possible input vectors. Further, every $X \in X^\infty$ is sampled uniformly at random with a probability of $Pr_{\mathbb{D}X^\infty}[X] = \frac{1}{16}$. We limit the set of possible fault transformations to $\mathcal{T} = \{\tau^{\text{set}}\}$. Therefore, each fault is given by $\psi = (g, \tau^{\text{set}})$ with $g \in G_C$ while Ψ^∞ denotes the power set of Ψ with $|\Phi^\infty| = 2^{|G_C|} = 8$. While FIESTA supports arbitrary probability distributions $\mathbb{D}\Phi^\infty$ over Φ^∞ , this example focuses on a uniform distribution of the fault combinations. Specifically, we assume that an independent fault occurs with $\varphi = \frac{1}{2}$. Therefore, each fault combination $\Psi \in \Phi^\infty$ occurs with probability

$$Pr_{\mathbb{D}\Phi^\infty}[\Psi] = \varphi^{|\Psi|} \cdot (1 - \varphi)^{|G_C| - |\Psi|} = \left(\frac{1}{2}\right)^3 = \frac{1}{8}.$$

Using the previously mentioned characteristics of a random fault adversary $\mathcal{A}^\zeta$ we formally define the adversary studied in this example by $\zeta = (\frac{1}{2}, \Psi^\infty, 0)$. To exhaustively determine the advantage of the adversary $\mu = Adv(\mathcal{A}^\zeta)$, it is necessary to compute Equation (5). Since $Pr_{\mathbb{D}\Phi^\infty}[\Psi]$ and $Pr_{\mathbb{D}X^\infty}[X]$ are constant, Equation (5) can be simplified to

$$\mu = \frac{1}{8} \cdot \frac{1}{16} \sum_{\Psi \in \Phi^\infty} \sum_{X \in X^\infty} E(C, \Psi, X).$$

It remains to evaluate $E(C, \Psi, X)$ as defined in Equation (6) for all $(\Psi, X) \in \Phi^\infty \times X^\infty$. Since $E(C, \Psi, X) = 1$ if Ψ is effective w.r.t. X and is otherwise zero, we can simply count the number of effective faults for all combinations of $X \in X^\infty$ and $\Psi \in \Phi^\infty$. The results are illustrated in Table 2, where a value of one indicates the presence of $\psi \in \Psi$. To compute the number of effective faults, the difference $\Delta = y \oplus \tilde{y}$ between the intended output y of the golden circuit and the retrieved output $\tilde{y}$ of the faulted circuit is calculated for all $X \in X^\infty$. The number of effective faults is then given by the count of instances where $\Delta = 1$.

Table 2 reports a total number of 70 effective faults while the remaining 58 faults turned out to be ineffective. Therefore, $\mu = Adv(\mathcal{A}^\zeta)$ is computed by

$$\mu = \frac{1}{8} \cdot \frac{1}{16} \cdot 70 = 0.546875.$$

Table 2: Overview of the number of effective and ineffective faults.

Induced Faults			Counts	
(g_1, τ^{set})	(g_2, τ^{set})	(g_3, τ^{set})	effective	ineffective
0	0	0	0	16
1	0	0	12	4
0	1	0	12	4
0	0	1	10	6
1	1	0	6	10
1	0	1	10	6
0	1	1	10	6
1	1	1	10	6
			70	58

Next, we demonstrate how μ can be approximated with (μ^ℓ, μ^u) by randomly selecting half of the combinations from $X \in X^\infty$ and $\Psi \in \Phi^\infty$, turning the evaluation into an incomplete setting. In this example, we assume that 33 effective faults are randomly sampled, while 31 sampled faults are ineffective. Both counts are used to solve Equation (11) numerically for a confidence level of $\gamma = 1 - 10^{-5}$, resulting in $\mu^\ell = 0.25093324$ and $\mu^u = 0.77418432$. For this small number of observations and the specified confidence level, the distance between the upper and lower bounds of the confidence interval is larger than 0.5, indicating an imprecise estimation.

3.6.1 Exhaustive Computation vs. FIESTA Approximation

Based on the above example, we demonstrate how the size of the confidence interval evolves with an increasing number of simulations. To evaluate the above example with FIESTA, C was implemented in Verilog and synthesized with Yosys [Wol]. Using FIESTA, we automatically determine the confidence interval $[\mu^\ell, \mu^u]$ using a varying number of simulations. Both boundaries, μ^ℓ and μ^u , are depicted in Figure 3 for confidence level $\gamma = 1 - 10^{-5}$. All other parameters are configured as defined in Section 3.6. We recall that for each simulation, both the input vector $X \in X^\infty$ and the fault combination $\Psi \in \Phi^\infty$ are sampled identically and independently. The sampling process is not adjusted by the fact that all elements of the set could be exhaustively enumerated. Therefore, the confidence interval still exhibits a large imprecision even after $|X^\infty| \times |\Phi^\infty| = 2^7$ simulations, despite this number being sufficient to exhaustively determine the advantage, or in other words to achieve full fault coverage. The red horizontal line represents the previously computed exact advantage $\mu = 0.546875$. It is evident that the confidence interval decreases as the number of simulations increases. For $\log_2(n) = 6$ simulations, the interval is $(0.2509, 0.7741)$. However, it narrows to $(0.5461, 0.5504)$ for $\log_2(n) = 20$, where each of the 2^7 input-fault combinations could be investigated multiple times.

Due to the random selection of fault combinations in the simulations, we can only provide bounds for the fault coverage. We do not track how many different faults $\Psi \in \Phi$ are induced, as this tracking incurs a cost. However, this would be required to determine the exact fault coverage. We have optimized FIESTA to analyze even large circuits, where any additional unnecessary cost should be avoided. The upper bound is equal to the number of simulations n . The lower bound corresponds to a single fault combination, assuming that the same fault combination, $\Psi \in \Phi^\infty$, is sampled in all simulations.

The computation for $\log_2(n) = 20$ took 0.2s on a laptop with 32 GB RAM and an Intel i7-1365U Central Processing Unit (CPU).

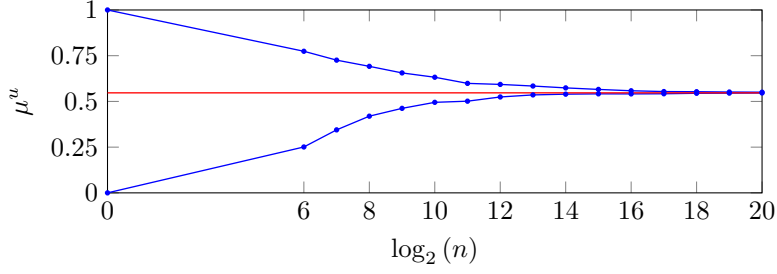


Figure 3: Comparison of the confidence interval (μ^ℓ, μ^u) computed with FIESTA and the analytically derived exact advantage μ .

4 Comparison to Related Work

In this section, we compare our approach taken for FIESTA with existing tools from the literature that verify fault security notions. By doing so, we identify limitations but also advantages of FIESTA.

4.1 VERICA⁺

VERICA⁺ is an open-source framework able to verify combined random probing security [BFG⁺24] using Binary Decision Diagrams (BDDs) for symbolic simulation. In addition, it enables the independent evaluation of general random fault security, based on the approach originally introduced by the FIVER framework [RSS⁺21]. FIVER abstracts a circuit, provided as a gate-level netlist, into a BDD that can be modified with a user-defined fault combination $\Psi \in \Phi^\infty$. Then, FIVER compares all output nodes of the faulty BDD with the output nodes of a golden BDD, which remains unaltered by any fault combination. If two output nodes differ for at least a single input vector for any input assignment $X \in X^\infty$, indicating that the faulty BDD produces a different result than the golden BDD, an effective fault, according to [BFG⁺24], is identified. VERICA⁺ employs this strategy to evaluate all fault combinations $\Psi \in \Phi^\infty$ up to a threshold of $|\Psi| \leq \beta$. Fault combinations exceeding this limit are not analyzed, introducing uncertainty in the computed advantage. To mitigate this, VERICA⁺ determines both upper and lower bounds for the advantage. When an effective fault combination is detected, its probability is calculated and accumulated.

Let Φ^{eff} be the set of all evaluated $\Psi \in \Phi^\infty$ with $|\Psi| \leq \beta$ that are effective. VERICA⁺ computes the lower bound of the advantage as

$$\mu^\ell = \sum_{\Psi \in \Phi^{\text{eff}}} \Pr_{\mathbb{D}\Phi^\infty}[\Psi]. \quad (15)$$

For the upper bound, VERICA⁺ assumes that all non-evaluated fault combinations $\Psi \in \Phi^\infty$ with $|\Psi| > \beta$ are effective. Denoting the set of all non-evaluated fault combinations by $\hat{\Phi}$, the upper bound is computed as

$$\mu^u = \mu^\ell + \sum_{\Psi \in \hat{\Phi}} \Pr_{\mathbb{D}\Phi^\infty}[\Psi]. \quad (16)$$

4.2 IronMask⁺

IronMask [BMRT22] was originally developed to efficiently verify the security of cryptographic gadgets under the (random) probing model. The capabilities of IronMask were later expanded in IronMask⁺ [BFG⁺24] to incorporate fault tolerance, enabling the verification

of combined random security for gadgets. The GitHub repository for IronMask⁺⁵ includes a helper script designed to determine the effective fault combinations, which plays a crucial role in verifying the combined random security of these gadgets. Unlike VERICA⁺, which requires a gate-level netlist, IronMask⁺ expects a Sage file in a custom format. This format enables a mathematical description of a gadget, restricting operations to addition and multiplication. Moreover, IronMask⁺ and the helper script support only the set τ^{set} and reset τ^{reset} fault transformations, whereas VERICA⁺ also supports τ^{toggle} .

The helper script parses the Sage file into an internal symbolic representation and uses symbolic computation to identify fault combinations that lead to effective faults. It can be configured to identify only fault combinations $\Psi \in \Phi^\infty$ with $|\Psi| \leq \beta$ that are effective. For this purpose, the script generates a list of effective fault combinations $\Psi \in \Phi^\infty$ with $|\Psi| = x$ for each $x \leq \beta$ and writes each list to a separate file. The main part of IronMask⁺ then computes the upper bound μ^u and lower bound μ^ℓ of the advantage μ based on these lists using the technique explained in Section 4.1.

4.3 Comparison with FIESTA

FIESTA differs from VERICA⁺ and IronMask⁺ in terms of applicability, accuracy, and runtime. We compare FIESTA against VERICA⁺ and IronMask⁺ to identifying both its advantages and potential drawbacks relative to these tools.

4.3.1 Applicability

VERICA⁺ is limited to the verification of circuits that can be modeled as DAG, which means that VERICA⁺ cannot deal with circuits that contain loops. While unrolling can be used to remove loops from a circuit, analyzing fully unrolled block ciphers, such as AES, remains infeasible. Specifically, generating a BDD for such unrolled block ciphers is impractical due to their highly interconnected structure, which is essential for achieving sufficient diffusion [RSS⁺21, 5.2 Limitations]. However, VERICA⁺ can still process a single round of the AES. Similarly, the approach taken by IronMask⁺ restricts its verification capabilities to a specific class of gates, known as C(N)LR-gadgets. In contrast, FIESTA does not have such limitations; it is restricted to neither gadgets nor circuits that can be represented as a DAG. It can simulate arbitrary ciphers, such as the AES, and assess their security under the (general) random fault model. Furthermore, FIESTA is applicable to other cryptographic primitives beyond the AES as well as non-cryptographic primitives and even complex CPUs.

4.3.2 Accuracy

To achieve perfect accuracy, i.e., to ensure $\mu^\ell = \mu = \mu^u$, an exhaustive evaluation of both the inputs $X \in X^\infty$ and all fault combinations $\Psi \in \Phi^\infty$ is required. IronMask⁺ and VERICA⁺ exhaustively evaluate the effect of a fault Ψ for all $X \in X^\infty$ using symbolic computation or BDDs, respectively. While these exhaustive tools can evaluate all fault combinations, a threshold β can be configured to limit the evaluation to fault combinations with $|\Psi| < \beta$. This is necessary when it is computationally infeasible to evaluate all possible $\Psi \in \Phi^\infty$. If β is specified, the exhaustive tools are only exhaustive in the inputs $X \in X^\infty$ but not in the fault combinations $\Psi \in \Phi^\infty$. This introduces uncertainty, as fault combinations with $|\Psi| \geq \beta$ are not evaluated. Contrary, FIESTA is neither exhaustive in the inputs $X \in X^\infty$ nor in the fault combinations $\Psi \in \Phi^\infty$, introducing additional uncertainty in the confidence interval. When the exhaustive tools are computationally able to evaluate all $\Psi \in \Phi^\infty$, i.e., $\beta = |\Psi^{V_B}|$, they are always exact and compute $\mu^\ell = \mu = \mu^u$, as shown in the example in Section 3.6. In such cases, the accuracy of exhaustive tools is

⁵<https://github.com/CryptoExperts/IronMask/tree/fe3ecea730ff8cba014a0326285a3da5f63610ac>

always higher than that of FIESTA, regardless of the number of simulations performed. This is because FIESTA can only approximate μ due to its sampling-based nature, as shown in Section 3.6.1. Moreover, when FIESTA reports an interval $[\mu^\ell, \mu^u]$, it may be invalid with a user-defined miss probability $\alpha > 0$, whereas for all intervals reported by VERICA⁺ and IronMask⁺, it holds that $\alpha = 0$. Therefore, α must be set to a sufficiently small acceptable value. Prior to illustrating the accuracy of the tools under comparison, we note that their reported absolute bounds are not directly comparable due to the following reasons.

- Since IronMask⁺ analyzes a different representation of the circuit compared to the other tools, it may express a single gate as multiple operations, even though a practical Process Design Kit (PDK) would allow implementing it as a single gate. Hence, if an adversary can target all gates, an implementation with more gates may offer a higher advantage.
- Due to the differing definitions of effectiveness, as discussed in Section 2.4.3, the exhaustive tools and FIESTA yield identical bounds only if the circuit is designed such that each fault combination is either effective for all inputs or for none. In Section 3.6, the probability of one of the seven fault combinations, each containing at least one fault ($1 \leq i \leq 7$), is given by $Pr_{\mathbb{D}\Phi^\infty}[\Psi] = \frac{1}{8}$. The fault-free case occurs with a probability of $\frac{1}{8}$ as well. Each of these seven fault combination is drawn effective by at least one input, shown in Table 2. Therefore, the exhaustive tools compute the advantage of the adversary by $\mu = \frac{1}{8} \cdot 7 = 0.875$. However, as explained and used for the calculation in Section 3.6, our definition of effectiveness requires incorporating the probability of the input and yields a lower result ($\mu = 0.546875$).
- Our approach induces faults into gates $g \in G_C$ of a circuit execution C , as defined in Section 2.3. In contrast, the exhaustive tools fault the structural gates $v \in V_B$ of a structural circuit B . Specifically, our method allows faulting a gate $g \in G_C$ across any number of clock cycles during simulation. This means that a gate can be faulted in a specific cycle, every other cycle, all cycles, or any combination of cycles. The injected fault is present for only one cycle, as explained in Section 2.4. Conversely, the exhaustive tools are limited to faults that affect structural gates for the entire circuit evaluation, rather than just a single cycle.

Instead, we focus on the comparing the interval lengths $L = \mu^u - \mu^\ell$ to compare the accuracy. We target a single SININA gadget [DN20, RFSG22], which is also used in the case studies in [BFG⁺24]. This gadget can be verified by all three tools under comparison and was previously studied in [BFG⁺24]⁶. For FIESTA, we additionally assume that each structural gate can be targeted in all the cycles required to compute the final result. In the case of the SININA gadget, this involves three cycles and leads to $|\Psi^{G_C}| = 360$. The adversary can only apply the τ^{set} fault transformation. Thus, $|\Phi^\infty| = 2^{120}$ for VERICA⁺ and $|\Phi^\infty| = 2^{360}$ for FIESTA. The results of multiple experiments featuring VERICA⁺ and FIESTA are depicted in Figure 4. The red line represents a set of experiments performed by VERICA⁺ with $\beta = 1$, while the blue line represents experiments with $\beta = 6$, also calculated with VERICA⁺. Even with $\beta = 6$, only a small portion of the fault combinations is evaluated. Specifically, less than 2^{32} of the 2^{120} fault combinations are evaluated. We have limited our case studies to $\beta = 6$ because higher values of β cause an overflow in the VERICA⁺ code, leading to inaccurate results. We have notified the authors of VERICA⁺ about this issue. All transparent lines represent experiments computed by VERICA⁺ with $2 \leq \beta \leq 5$. As β increases, the red part of the color of the lines decreases while the blue

⁶The other four designs used for the case studies in [BFG⁺24] were not available at the time we conducted our experiments.

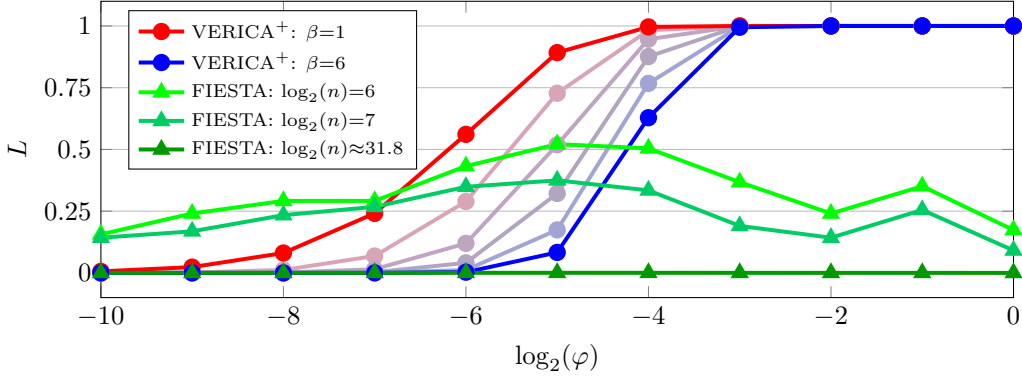


Figure 4: Comparison of the confidence interval length $L = \mu^u - \mu^l$ for VERICA⁺ and FIESTA across different probabilities φ .

part increases. Therefore, the second-highest line is for $\beta = 2$, and the second-lowest line is for $\beta = 5$.

Impact of the Simulation Count. For $\beta = 1$, L reaches 1 when $\log_2 \varphi = -4$, while for $\beta = 6$, this occurs only when $\log_2 \varphi = -3$. This trend suggests that as the number of evaluated fault combinations (β) increases, L decreases, thereby, enhancing accuracy. This trend is also observed with FIESTA. The three green lines represent the results of FIESTA for different numbers of simulations across all probabilities. The lightest green line corresponds to $n = 64$ simulations, the darkest to $n = 2^{31.842}$. The interval length L is highest for $n = 64$ across all probabilities and lowest for $n = 2^{31.842}$. Thus, both tools share the property of achieving shorter confidence intervals L with more processed fault combinations.

Impact of the Fault Probability. However, for VERICA⁺, L increases with growing φ . This shows that VERICA⁺ is particularly well-suited for analyzes where the majority of the cumulative probability of fault combinations is accounted for by combinations containing at most β faults. By expressing L using the CDF of the Binomial distribution it can be seen that the cumulative probability decreases with increasing φ . Specifically, assuming each fault occurs with probability φ , the probability p_i of a fault combination Ψ with $i = |\Psi|$ can be computed as $p_i = \varphi^i \cdot (1 - \varphi)^{|\Psi^{VB}| - i}$, where φ^i is the probability that i faults occur, and $(1 - \varphi)^{|\Psi^{VB}| - i}$ is the probability that the remaining $|\Psi^{VB}| - i$ faults do not occur. There are $\binom{|\Psi^{VB}|}{i}$ ways to choose i faults out of the possible $|\Psi^{VB}|$ faults. To compute $L = \mu^u - \mu^l$, the probabilities of all non-evaluated fault combinations must be accumulated, leading to

$$L = \sum_{\beta < i \leq |\Psi^{VB}|} \binom{|\Psi^{VB}|}{i} \cdot \varphi^i \cdot (1 - \varphi)^{|\Psi^{VB}| - i}.$$

This can be reformulated as

$$L = 1 - \sum_{0 < i \leq \beta} \binom{|\Psi^{VB}|}{i} \cdot \varphi^i \cdot (1 - \varphi)^{|\Psi^{VB}| - i}.$$

The right-hand sum represents the CDF of the Binomial distribution:

$$L = 1 - Pr_{(\varphi, |\Psi^{VB}|)}[\mathcal{B} \leq \beta], \quad (17)$$

which computes the probability of having β or fewer faults. A property of $Pr_{(\varphi, |\Psi^{V_B}|)}[\mathcal{B} \leq \beta]$ is that it decreases for constant β and $|\Psi^{V_B}|$ as φ increases because it becomes less likely to have at most β faults when each fault has a higher probability of occurring. Consequently, fault combinations with at most β faults accumulate to a higher probability for smaller φ , leading to a shorter interval length L . Conversely, the size of the confidence interval computed by FIESTA, as depicted by the three green lines with triangular marks, does not increase as φ increases. Instead, L increases when the ratio between the number of effective faults and ineffective faults approaches 1. In this scenario, μ approaches 0.5, maximizing the variance of the approximated binomial distribution and leading to an increased interval to capture the uncertainty of the estimate. For the analyzed SININA gadget, this is particularly evident when $\log_2(\varphi) = -5$, as shown by the light green line in Figure 4. However, the maximum length of the confidence interval computed by FIESTA, even with only 64 simulations (the minimum number that FIESTA supports), is approximately half of the maximum interval computed by VERICA⁺. This demonstrates that our statistical approach is less dependent on the chosen probability, even for small gadgets, making it more generic in that sense. However, VERICA⁺ is beneficial for small probabilities. It computes tighter bounds up to $\log_2(\varphi) = -7$ when considering only $\beta = 1$, while FIESTA conducts $n = 128$ simulations, as shown in Figure 4. We compare these two cases because VERICA⁺ evaluates 120 fault combinations for $\beta = 1$, whereas FIESTA simulates 128 randomly sampled fault combinations⁷. We repeat this experiment with $\beta = 6$ for VERICA⁺ and approximately $n \approx 2^{31.842}$ simulations for FIESTA. The figure clearly illustrates that FIESTA outperforms VERICA⁺ in terms of interval length L , provided that $\log_2(\varphi) > -6$. However, for smaller values of φ , the interval length L computed by VERICA⁺ is smaller than that computed by FIESTA. This indicates that exhaustive tools are preferable for evaluating small probabilities. Conversely, FIESTA has a clear advantage when dealing with larger φ .

Impact of the Number of Faults. Another trend evident from Equation (17) is that a higher number $|\Psi^{V_B}|$ of possible faults Ψ^{V_B} increases the length L of the confidence interval. Therefore, in Figure 4, we omit the interval length L for the results computed by IronMask⁺ to maintain the figure's clarity. Due to its custom format and the higher number of required gates, more faults $\psi \in \Psi^{V_B}$ are available to an adversary that targets all gates of a circuit. Consequently, the size L of the confidence interval for IronMask⁺ is larger than for VERICA⁺. This observation also impacts the accuracy of VERICA⁺. Not only is it more challenging to enumerate all fault combinations as the gate count increases, but the higher gate count also leads to less tight bounds with the strategy utilized by VERICA⁺. This is not the case for the strategy used by FIESTA. The only dependency of FIESTA on the setting of the problem is the real value of μ , as explained above.

Impact of the Confidence Level. The advantages of FIESTA come with a trade-off: it allows for a miss probability α that μ is not covered by the computed confidence interval. For the experiments above, $\alpha = 10^{-5}$ is assumed. With a probability of $\frac{\alpha}{2}$, μ is larger than μ^u , and with the same probability, μ is smaller than μ^ℓ . In contrast, for exhaustive tools, it always holds that $\mu^\ell \leq \mu \leq \mu^u$. The selected miss probability α , or equivalently the confidence level $\gamma = 1 - \alpha$, affects the length of the confidence interval L . A higher γ results in a larger L , assuming other parameters remain constant. This effect can be mitigated by increasing the number of processed simulations n .

⁷FIESTA can only perform multiple of 64 simulations, making 128 the closest number to 120.

4.3.3 Efficiency

For increasing circuit sizes the runtime of FIESTA primarily depends on the size of the circuit execution C , while the runtime of the exhaustive tools is primarily dependent on the complexity of the circuit. Nevertheless, the runtime of the exhaustive tools also scales with the size of the circuit, but the complexity of the circuit is dominant. This is in particular discussed in [RSS⁺21] for VERICA⁺. While IronMask⁺ focuses on specific gadgets as discussed in [BFG⁺24] and is therefore not able to analyze arbitrary complex circuits. Therefore, complex circuits such as full unrolled implementations of AES cannot be processed by the exhaustive tools. For constant circuits B or circuit executions C , respectively the runtime of the exhaustive tools scales with β , while the runtime of FIESTA scales with the number of performed simulations n . The runtime of FIESTA for a given circuit execution C depends on the selected number of simulations n that should be performed. In the following, we compare the efficiency of the exhaustive tool with that of FIESTA for three different classes of circuits, defined by their size and complexity. We do not use fault coverage as a metric here, since FIESTA does not track the number of different fault combinations $\Psi \in \Phi$ that are induced, as discussed in Section 3.6.1.

Exhaustive Evaluation. The exhaustive tools are more efficient than FIESTA when it is feasible to enumerate all possible fault combinations. This efficiency arises because exhaustive tools can utilize the knowledge that all fault combinations are evaluated, whereas the sampling-based strategy of FIESTA cannot incorporate this information. An example of such a situation is presented in Section 3.6.1, where VERICA⁺ determines the advantage exhaustively in 0.033 seconds.

Non-Exhaustive Evaluation. For combinations of adversaries and gadgets where evaluating all $|\Phi^\infty|$ faults is infeasible, as in Section 4.3.2, both the exhaustive tools and FIESTA compute approximated results. We examine the example from Section 4.3.2 concerning efficiency and illustrate the results in Figure 5. The runtime for approximating

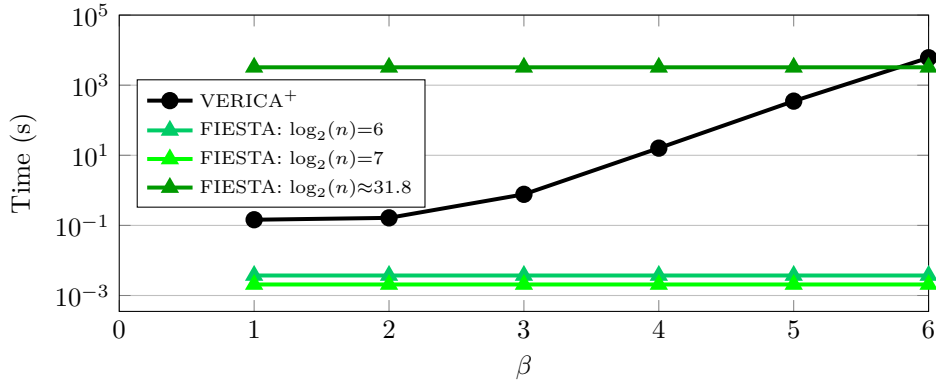


Figure 5: Runtime of VERICA⁺ for given β and runtime of FIESTA for different simulation counts n .

the advantage of the adversary for an arbitrary φ using VERICA⁺ depends on the chosen β . For instance, the approximation with $\beta = 1$ was computed in 0.145 seconds, while it took approximately 1.7 hours for $\beta = 6$. This exponential growth in runtime with respect to β for VERICA⁺ is depicted by the black line. Note that the runtime of both exhaustive tools is independent of the probability distribution, provided that the distribution allows for the same number of fault combinations, or equivalently, has the same number of non-zero

probabilities. The current version of VERICA⁺⁸ used in the experiments can evaluate up to $|\Phi^\infty| = 2^{32} - 1$ fault combinations. In contrast, FIESTA is only limited by the number of effective faults determined, allowing up to $2^{64} - 1$ effective faults before an overflow occurs. The runtime of FIESTA for $n = 2^m$ with $m \in \{6, 7, 31.842\}$ is depicted by the three green lines with triangular marks analogous to Figure 4. Since the runtime does not depend on β , the results of the three computations from Figure 4 are shown as constants. The lightest green line represents the runtime for $n = 64$ simulations, the darkest for $n = 2^{31.842}$ simulations, and the remaining line for $n = 128$. Figure 5 shows that FIESTA generally has a shorter runtime for approximately the same number of evaluated fault combinations. For the maximum of $n = 31.842$ simulations and $\beta = 6$ FIESTA is 1.87 times faster than VERICA⁺. The relation between runtime and accuracy depends on the specific evaluation. For $\log_2(\varphi) \leq -8$ the accuracy of VERICA⁺ is more than two times better, for $\log_2(\varphi) = -7$ the accuracy of both tools is approximately equal, while for $\log_2(\varphi) \geq -6$ FIESTA computes already a two times more accurate result. Consequently, in our example, FIESTA is more efficient regarding the accuracy if $\log_2(\varphi) > -8$. We emphasize again that the comparison must be taken with care due to the different definitions of effectiveness as explained in Section 2.4.3. We have omitted IronMask⁺ in Figure 5 because after more than 10 days and 6 hours, IronMask⁺ has not completed the computation of all fault combinations with $\beta = 5$. For $\beta = 4$, IronMask⁺ took approximately 8 hours, making it 1792 times slower than VERICA⁺, which completed the task in just 16 seconds. However, it is important to note that IronMask⁺ processes significantly more fault combinations due to its internal representation, which supports fewer gate types. This results in a higher gate count to represent a circuit. Specifically, IronMask⁺ must process $|\Psi^{VB}| = 232$ gates, leading to $|\Phi^\infty| = 2^{232}$ possible fault combinations, compared to $|\Phi^\infty| = 2^{120}$ for VERICA⁺. Consequently, IronMask⁺ is less efficient than VERICA⁺ for arbitrary evaluations in the general random fault model.

A common advantage of the exhaustive tools is their ability to first determine the effective fault combinations Φ_i^{eff} for all $1 \leq i \leq \beta$ with $i = |\Psi|$. This step is independent of the probability distributions of fault occurrences. Only in the second step these tools compute the bounds by applying Equation (15) and Equation (16). While the exhaustive tools currently compute both steps, it is theoretically possible to modify them to compute the first step only once when investigating a design-adversary combination for different probabilities. This modification can significantly enhance the generation of case studies. We employed this procedure to accelerate the creation of Figure 4. This speedup is not possible with FIESTA, as it always requires sampling the fault combinations $\Phi \in \Phi^\infty$ and the input $X \in X^\infty$ according to their probability distribution. However, this sampling process and simulation-based evaluation approach enable FIESTA to evaluate practical circuits that cannot be analyzed using the exhaustive tools.

Computationally Infeasible Evaluations of Exhaustive Tools. To the best of our knowledge, FIESTA is the only tool capable of analyzing complex circuits and arbitrary adversaries which are computationally infeasible for exhaustive tools. FIESTA does not need to simulate all inputs or fault combinations to compute a reasonable bound on the advantage, as discussed in Section 4.3.2. This is possible due to the statistical approach detailed in Section 3.4. However, the runtime of FIESTA depends on the probability distribution of the fault combinations. For simplicity, we assume that the faults are independent and equally likely with probability φ , as in the example from Section 4.3.2. Then, the runtime increases with the probability φ of injected faults Ψ , as depicted in Figure 6 for the 1-1-SININA gadget and 100 million simulations. The reason is that the probability φ determines the number of faults injected in each simulation. Higher probabilities lead to

⁸Commit: 4afb95, <https://github.com/Chair-for-Security-Engineering/VERICA/tree/4afb95ffb976c11315f00452b81962b2521c551>

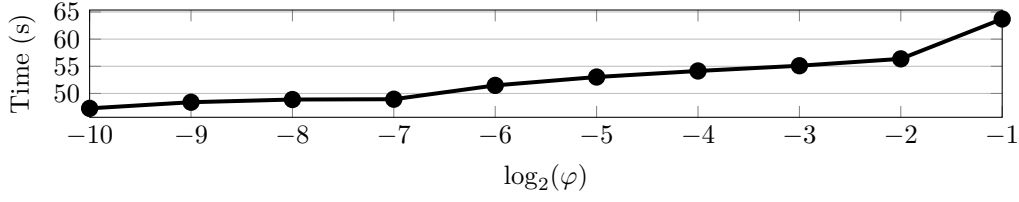


Figure 6: Runtime of FIESTA for 100 million simulation across different φ .

more faults being sampled, and the process of computing the mapping function $MG_C \rightarrow \mathcal{T}$, explained in Section 3.2, takes more time. Each gate is initially mapped to the identity fault transformation τ^{id} and is only overwritten when a fault is sampled. Additionally, each gate $g \in G_C$ is first evaluated correctly and then, if faulted, the fault transformation is applied, which takes additional time. This explains why higher fault probabilities φ lead to increased execution time.

5 Case Studies

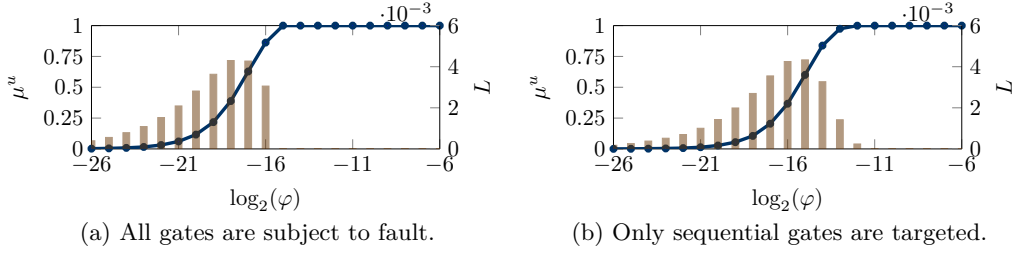
To demonstrate the gate-level evaluation procedure of the FIESTA framework, we consider practical implementations from the literature that are computationally infeasible to evaluate exhaustively using VERICA⁺ or IronMask⁺. Specifically, we target byte-serial implementations of the AES encryption function with a 128-bit secret key, hereafter referred to as **AES-128** [DR02], and investigate their security in the general random fault model. Some target implementations are equipped with different countermeasures against physical attacks, including masking, replication, and error-correcting codes. For practical reasons, we assume independent faults injected with probability φ and an imprecise adversary with no control over the location and number of faults that occur, analogous to [BFG⁺24].

5.1 Setup

We synthesized the designs used in Section 5.2 using the NanGate45 Open Cell Library and Synopsis Design Compiler. We run FIESTA on a docker container based on Ubuntu 22.04 running on a machine with two Intel(R) Xeon(R) Gold 5320 CPUs at 2.2 GHz and 2 TB of physical RAM. To evaluate our case studies, we utilized 52 logical cores through hyper-threading.

5.2 Evaluations

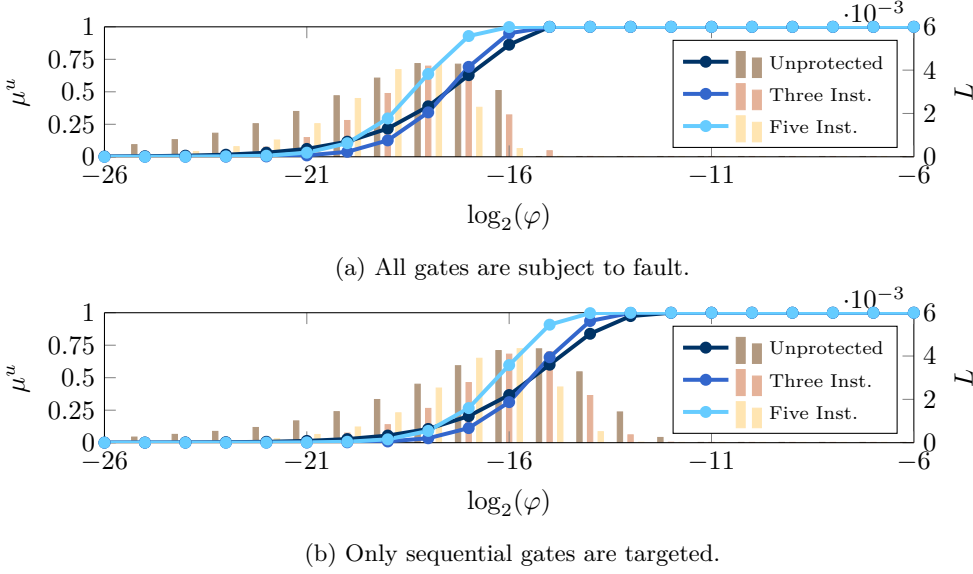
As a baseline, we used FIESTA to evaluate an unprotected **AES-128** core to assess its fundamental resistance to a weak adversary without control over the location or the number of induced faults. More precisely, we performed one million simulations with random inputs and faults, varying the fault probability φ but keeping a constant miss probability $\alpha = 10^{-5}$. We only consider adversaries utilizing stuck-at-zero faults, i.e., $\mathcal{T} = \{\tau^{\text{reset}}\}$, to allow the comparison of the same adversary against implementations using different countermeasures. The results, illustrated in Figure 7, are shown for two scenarios: (1) when all gates are considered as potential target for fault injections and (2) when only sequential gates are allowed to be faulty. In this and the following figures, we visualize the upper bound μ^u of the confidence interval for different fault probabilities by a blue line. Specifically, every dot on the blue line represents a μ^u shown on the left y -axis, corresponding to a specific fault probability φ on the x -axis. In addition, we visualize the length of the confidence

Figure 7: **AES-128** unprotected.

interval $L = \mu^u - \mu^\ell$, using bars which represent the difference between the upper and lower bounds for a specific φ , shown on the right y -axis. As expected, Figure 7 confirms that μ^u grows exponentially with φ meaning that FIESTA detects more effective faults with higher fault probabilities. From Figure 7a, the blue line indicates that even for the lowest investigated fault probability $\varphi \geq 2^{-26}$ a few effective faults are detected by FIESTA. Given that this unprotected **AES-128** core consists of approximately 1720 gates and operates for 230 clock cycles to accomplish an encryption, this corresponds to an overall fault injection probability of approximately 0.6% per simulation. Consequently, we expect faults in roughly 6000 out of one million simulations. However, FIESTA reports only around 1472 effective faults, indicating that the majority of the injected faults are likely ineffective. Further, for $\varphi \geq 2^{-15}$, all faults investigated by FIESTA are effective, indicated by $\mu^u = 1$ for $\varphi = 2^{-15}$ (see the corresponding blue dot). In this scenario, more than 10 faults per simulation are expected, which an unprotected **AES-128** core is unable to mitigate without additional countermeasures. Between these two extremes, FIESTA dynamically adjusts the size of the confidence intervals to account for increased uncertainty. The maximum size of the confidence interval is reached for $\varphi = 2^{-18}$, as indicated by the tallest bar. This is the case as for $\varphi = 2^{-18}$, the number of effective and ineffective faults are nearly equal, meaning that FIESTA cannot confidently determine the outcome based on an incomplete evaluation. A similar behavior is observed when only sequential gates can be faulty, as shown in Figure 7b. However, the upper bounds are slightly lower compared to Figure 7a, since fewer gates are susceptible to faults. This can be observed by comparing the blue lines in Figure 7a and Figure 7b. Comparison of the tallest bars in both figures reveals that the maximum confidence interval in Figure 7b occurs at a higher fault probability of $\varphi = 2^{-17}$ compared to Figure 7a.

5.2.1 Influence of Replication

To evaluate a first protection mechanism, we replicated the above **AES-128** and employed majority voting to correct faults. Specifically, we implemented three (resp. five) instances of the **AES-128** core while correcting faults with a majority voting module at the primary outputs. In the three-instance case, the majority voting mechanism can correct a single fault per execution. In the five-instance case, it can handle up to two faults per execution. The parameters used to evaluate these implementations were kept consistent with those used for the unprotected **AES-128** design. The evaluation results for the fault-tolerant implementations, along with those of the unprotected design for comparison, are presented in Figure 8. Additionally, it can be observed that implementing a majority voting mechanism capable of correcting a single fault slightly enhances protection against an imprecise adversary when φ is low, specifically in the range of 2^{-26} to 2^{-18} . For these low probabilities, the adversary's success rate is slightly reduced compared to the unprotected design. This improvement can be attributed to the limited number of faults that can be injected, which are effectively corrected by the majority voting mechanism. However, for a larger φ , ranging from 2^{-17} to 2^{-15} , the same design is even more susceptible to the considered adversary than the unprotected design. At higher probabilities, multiple faults

Figure 8: Comparison of **AES-128** unprotected with **AES-128** replicated multiple times.

are injected per simulation, exceeding the correction capability of the employed majority voting mechanism. This vulnerability is especially critical when the same fault - or, more precisely, two faults affecting the same output bit - is induced in both instances of the **AES-128** core, rendering majority voting ineffective. The same trend is observable when increasing the replication to five instances capable of correcting two faults per execution. Specifically, when only sequential gates are impacted and $\varphi < 2^{-20}$, the five-instance replication offers only a marginal improvement in resistance compared to the three-instance replication. In all other cases, it is more susceptible to faults than the three-instance variant. In general, majority voting provides only marginal improvements over the unprotected design when faults are unlikely, as such protection mechanisms can correct only a limited number of faults. However, for higher fault probabilities, replication can negatively impact fault tolerance.

5.2.2 Influence of Impeccable Circuits II (ICII)

A more advanced method for error correction in hardware circuits is described in [SRM20]. This technique, known as ICII, leverages an error-correction code and effectively limits the propagation of injected faults within the circuit, ensuring the correction of a specific number of faults per clock cycle. To protect our target **AES-128** design, we decided to utilize the AGEFA tool, which automates the transformation of an unprotected Verilog netlist into a design protected by Impeccable Circuits II [MM24]. We took the resulting design which is publicly available on GitHub⁹. The target design, with analysis results shown in Figure 9, employs an error-correcting code capable of correcting a single-bit fault (due to the application of an error-correcting code with a minimum distance $\delta = 3$) in four-bit chunks of the cipher state. This approach encodes each message into an eight-bit codeword, comprising the original four-bit message and four parity bits. We selected this design since – based on [MM24] – it is supposed to be a comparably small implementation compared to the replicated design with the same error correction capability. We should mention that two different designs with the same configuration are presented in [MM24] (and in the aforementioned GitHub repository), one with and another one without decomposing the

⁹https://github.com/Chair-for-Security-Engineering/AGEFA/tree/main/CaseStudies/07_AES128_byte_serial_encryption/netlists/AES_IC2_w_dec_d1/k4

round function. Furthermore, we chose the decomposed design to use two correction layers instead of a single one allowing a more efficient limitation of the fault propagation.

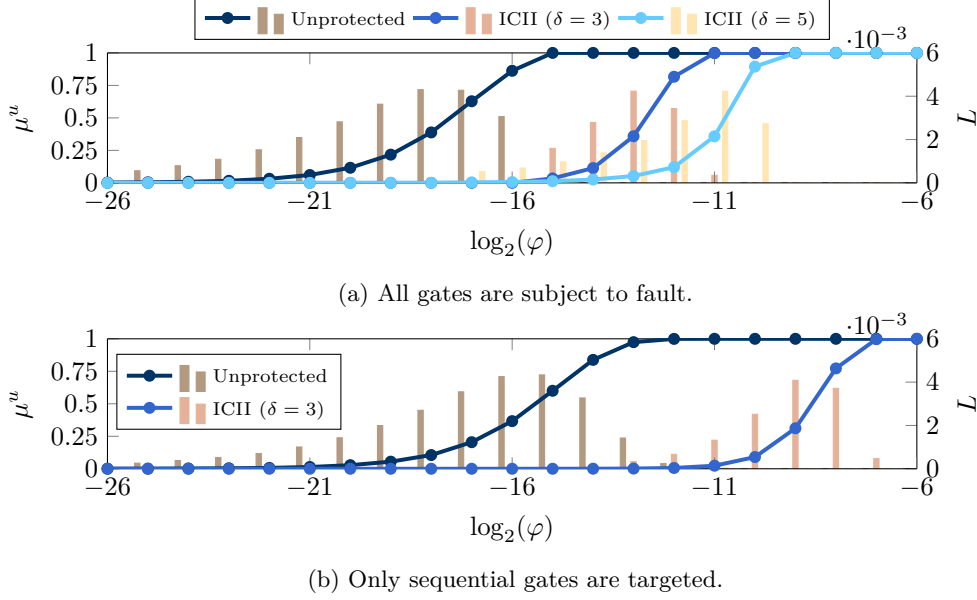


Figure 9: Comparison of **AES-128** unprotected with **AES-128** protected with Impeccable Circuits II.

Although the guarantees for this design are limited to correcting single-bit faults, our results, depicted in Figure 9, show that a significant number of randomized fault vectors can also be corrected. Consequently, a higher fault probability is required to successfully inject faults compared to the unprotected design. For instance, while a fault probability of $\varphi = 2^{-18}$ results in approximately 38% effective faults in the unprotected design, the protected design reduces this to around 35% for a much lower fault probability of $\varphi = 2^{-13}$. Furthermore, we consider a similar design protected with ICII, which employs an error-correcting code with a minimum distance of $\delta = 3$, effectively ensuring the correction of up to two-bit faults per codeword¹⁰. Figure 9a illustrates that increasing the minimum distance from 3 to 5 again increases the protection significantly. Specifically, the upper bound reached for the implementation with $\delta = 3$ and $\varphi = 2^{-13}$ is comparable to that for $\delta = 5$ and $\varphi = 2^{-11}$, indicating that the design with $\delta = 5$ can tolerate a fault probability that is four times higher compared to $\delta = 3$. However, this improvement is smaller compared to the gain achieved by the protected implementation with $\delta = 3$ over the unprotected one.

5.2.3 Influence of Masking

To evaluate the impact of masking, a prominent countermeasure against Side-Channel Analysis (SCA) attacks, on the resilience to faults, we automatically masked the previously targeted **AES-128** implementation using AGEMA [KMMS22]. Specifically, AGEMA replaces each gate in the circuit with composable masked gates, known as gadgets. In this case, we used low-latency HPC3 gadgets [KM22] to create the masked circuits. In fact, we have taken the first- and second-order masked designs, which are publicly available on

¹⁰We generate the evaluation results for all cells only, due to the high runtime of FIESTA on such a large design.

[GitHub](#)¹¹. As follows, we illustrate the evaluation results for the first- and second-order masked design (cf. Figure 10). Both figures demonstrate that masking negatively impacts

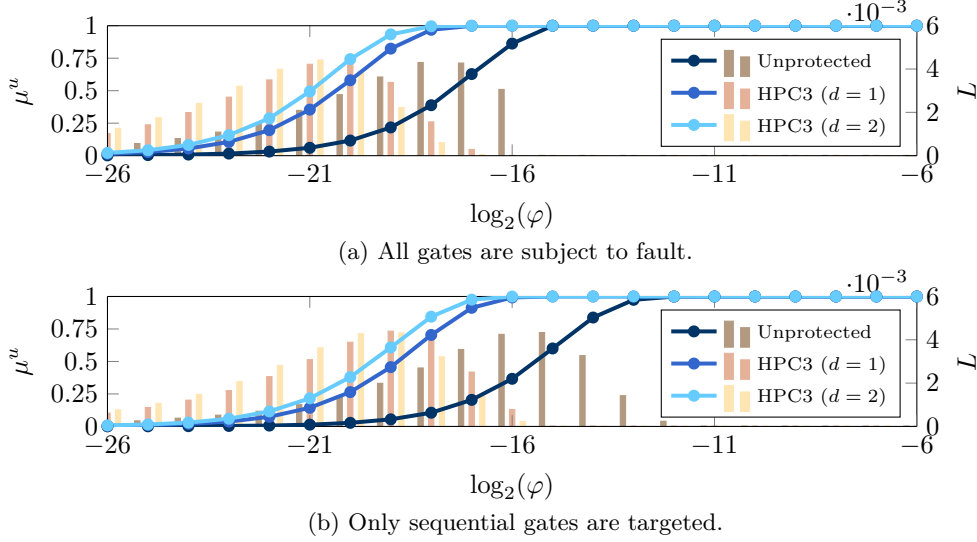


Figure 10: Comparison of **AES-128** unprotected with masked **AES-128** protected with first-order (resp. second-order) HPC3 gadgets.

resilience against faults. Specifically, for a first-order masked design, similar upper bounds to those of the unprotected design are achieved with a value of φ eight times smaller. This behavior can likely be attributed to the larger circuit size of the masked implementations, which increases the expected number of faults without enhancing fault resilience. In contrast, the second-order masked design is only slightly more susceptible to faults than the first-order design. This trend suggests that as the security order and circuit size increase, the susceptibility to faults also grows.

5.2.4 Conclusion

We conclude that both qualitative and quantitative criteria influence the fault resilience of a circuit. The quantitative criterion pertains to the number of gates that can be targeted by an adversary. A higher number of gates increases the likelihood of successful faults, thereby enhancing the adversary’s advantage. This effect is illustrated in Section 5.2.1, where the AES design protected using replication performs worse than the unprotected core under higher fault probabilities. The reason is that the increased number of gates is not sufficiently compensated by the qualitative criterion, that is, the effectiveness of the logic in resisting faults. The qualitative strength of the countermeasure in the studied model appears to be compromised by known practical weaknesses. In particular, when a simple majority vote is employed, faults can still propagate through the circuit, as discussed in [AMR⁺20, SRM20]. In contrast, Impeccable Circuits II, evaluated in Section 5.2.2, demonstrates stronger protection across all fault probabilities. Notably, even its second-order protection offers more robust security guarantees than first-order protection. In this case, the qualitative gains provided by the circuit’s design more than compensate the quantitative increase in gate count, resulting in a net improvement in fault resilience. We attribute the superior performance of this scheme to its ability to prevent fault propagation. Furthermore, we hypothesize that these insights into how protection mechanisms scale with different protection orders may extend to practical attack scenarios.

¹¹https://github.com/Chair-for-Security-Engineering/AGEMA/tree/main/CaseStudies/07_AES128_byte_serial_encryption/netlists

Specifically, if a circuit has more gates, it presents more opportunities for an adversary to inject faults. If the added complexity is not matched by an equally robust protection strategy, as is achieved with Impeccable Circuits II but not with simple majority voting, then it is reasonable to expect that the former will offer better real-world fault tolerance. This mirrors our observations under the general random fault model. In general, the absolute adversarial advantage for a specific circuit may not offer meaningful insight in isolation. More informative is how this advantage scales with increasing protection levels, and how that scaling compares across different protection strategies. Such analysis can help to determine whether the additional cost of higher-order protection is justified. If stronger protection leads to increased adversarial advantage across a wide range of attack models, the additional complexity and resource cost may not be warranted. However, this hypothesis requires validation through practical, implementation-level measurements, which we leave for future work.

6 Physical-Level Verification

Rather than manually defining a set of n^4 different adversaries, we present an extension to FIESTA, which enables the automated derivation of local fault combinations using physical information about the circuit under investigation. The incorporated physical information is provided as a GDSII file implemented using the SKY130 high-density digital standard cell library¹², since SKY130 is the only cell library currently supported by FIESTA, which can be physically implemented¹³. The GDSII file encompasses the layout information of the targeted Integrated Circuit (IC). Based on the given configuration, the extension can extract all gates of a specific type within a specified area. This approach enables the automatic definition of adversaries that affect only structural gates within a specific limited area. As explained in Section 2.2, laser fault injections as well as EMFI exhibit such local effects, which can be traced to particular areas. Therefore, this extension of FIESTA can be used to automate the modeling of adversaries utilizing these fault injection facilities. For global effects, as explained in Section 2.2, the extension provides no benefit. Instead, it remains the user’s responsibility to model the physical effects manually. However, once modeled manually, FIESTA can verify a circuit’s execution against adversaries utilizing such global effects. Nevertheless, the fully automated flow enabled by the extension explained below cannot be utilized in this case.

6.1 Skywater 130 Standard Cell Library (SCL) Integration

To (1) simulate manufacturable circuits and (2) analyze the impact of gate location on the effectiveness of EMFI, we integrate the `sky130_fd_sc_hd` Standard Cell Library (SCL) from the SkyWater 130 PDK into our simulation workflow. For brevity, we refer to this SCL as SKYwater 130 (SKY130). This library is publicly accessible through an archived [GitHub repository](https://github.com/google/skywater-pdk-libs-sky130_fd_sc_hd)¹⁴. As previously mentioned, the simulator requires a library file that defines the functionality of each gate in the gate-level netlist. This is the only requirement for adapting the simulator to a different SCL. We used a Python script to automatically generate a library file for SKY130 by extracting all necessary gates from the aforementioned GitHub repository. The gates in SKY130 are categorized into three groups:

Combinational gates are stateless components, whose functionality can be abstracted by a Boolean function operating solely on their inputs. Since combinational gates are fundamental to our circuit model (see Definition 2), they must be included in the

¹²`sky130_fd_sc_hd`, which is publicly available via GitHub: https://github.com/google/skywater-pdk-libs-sky130_fd_sc_hd.

¹³The NanGate45 Open Cell Library cannot be implemented to a GDSII file.

¹⁴https://github.com/google/skywater-pdk-libs-sky130_fd_sc_hd

simulator’s library file.

Sequential gates compute their output based on the input signals and an internal logic state. We model such a state by abstracting sequential gates with a Boolean function depending on the gate’s inputs and current output. Since sequential gates are fundamental to our circuit model (see Definition 3), they must be included in the simulator’s library file. The simulator supports only synchronous, positive-edge-triggered sequential elements¹⁵.

Physical level gates (like VDD and GND pads) are excluded from our circuit model and, consequently, are not included in the simulator’s cell library.

6.2 Adversary Generation using Physical Layout Information

We parse the GDSII file describing the physical representation of a circuit B using GDSII Tool Kit (Gdstk) [Gab20]. Gdstk represents the GDSII file with structures that store information about the physical cells $c_v \in \mathcal{C}_{V_B}$ implementing gates $v \in V_B$. These structures include the cell’s name n_{c_v} , type t_{c_v} , and bounding box $b_{c_v} = ((x^\ell, y^\ell), (x^u, y^u))$ of the gate. A simplified visualization of the bounding boxes b_{c_v} and $b'_{c'_v}$ for cells c_v and c'_v using schematic symbols instead of visualization of the physical representation, is shown in Figure 11. A bounding box encodes both the dimensions and the position of the physical

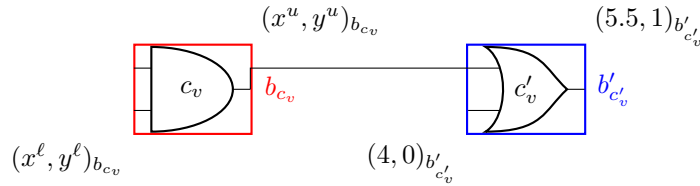


Figure 11: Simplified Representation: Bounding box b_{c_v} and $b'_{c'_v}$ for the physical layout of cell c_v and c'_v using schematic symbol.

representation of a gate. The dimensions are given by the length $\ell^y = y^u - y^\ell$ and the width $\ell^x = x^u - x^\ell$. However, since a bounding box is specified by only two coordinates, it cannot model all possible shapes of a physical gate. Therefore, the bounding box of the cell c_v corresponding to a gate $v \in V_B$ is defined as the smallest rectangle that ensures the physical representation of c_v fully fits within it, as illustrated in Figure 11. The bounding boxes are crucial for our automated adversary generation approach, as they determine which structural gates $v \in V_B$ can be attacked simultaneously by an adversary. Note that Gdstk extracts all cells from the GDSII file, including those classified as *physical level gates* in Section 2.3, which are not relevant for our approach. To address this, we have implemented a blacklist containing the names of these physical level gate types. After parsing the GDSII file with Gdstk, we iterate over the list of all cells, extracting only those that are non-physical (i.e., sequential or combinational). The remaining cells $c_v \in \mathcal{C}_{V_B}$ can then be represented by a structural gate $v \in V_B$ in our circuit model.

The remaining step is to identify which structural gates $v \in V_B$ can be targeted simultaneously by a specified adversary. To do this, we model an area A where the local effect utilized by the adversary $\mathcal{A}^\zeta$ manifests. We assume that all cells $c_v \in \mathcal{C}_{V_B}$ within area A are faulted. Specifically, we assume that it is sufficient when c_v has an overlap with A . We implement the area as a tuple (r_A^p, R_A^s) , where r_A^p is the primary bounding box and R_A^s is a list of secondary bounding boxes. The primary bounding box r_A^p is analogous to the bounding box of a cell, representing the smallest rectangle that fully covers the

¹⁵The simulator can be extended to cover other sequential gates as well, but this does not affect the functionality and concept of FIESTA.

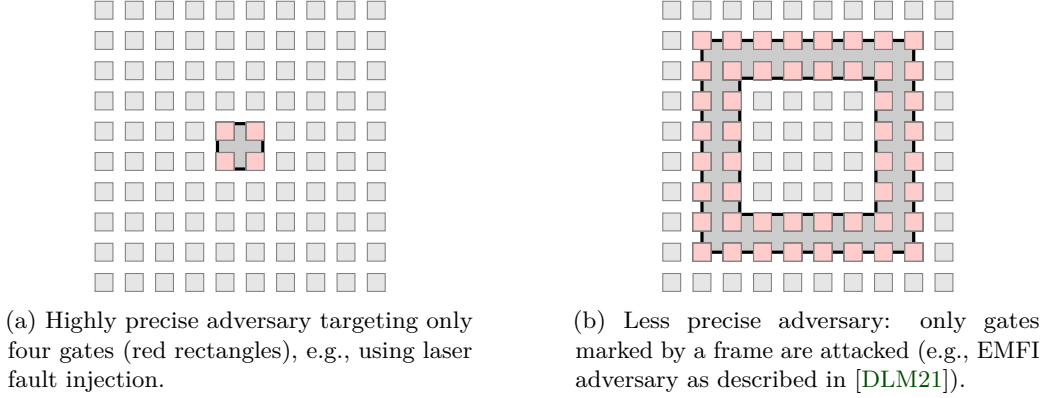


Figure 12: Overview of two different adversaries: the dark gray rectangle depicts the affected area, light gray rectangles represent physical cells. The red rectangles mark gates, which can be attacked simultaneously by one adversary, utilizing the visualized shape.

area affected by a local effect, as illustrated in Figure 12a. This example assumes a highly precise adversary, whose local effect is limited to the gray rectangle with black borders. Thus, the area corresponding to this adversary can be defined by $A = (r_A^p, \emptyset)$, where r_A^p describes that rectangle. The list of secondary bounding boxes R_A^s is only required if more complex shapes of adversarial effects need to be modeled. An example is depicted in Figure 12b, where the area A is modeled by tuple (r_A^p, R_A^s) . Here, r_A^p is defined by the black border of the outer rectangle and R_A^s contains one additional bounding box $r_A^{s1} \in R_A^s$, defined by the inner rectangle with black borders. The secondary bounding boxes $r_A^s \in R_A^s$ are used to exclude parts of the area covered by r_A^p from the area A impacted by the local effect. Consequently, these bounding boxes represent the unaffected area within r_A^p . By combining R_A^s and r_A^p to model A we can represent an arbitrary area where the local effect utilized by the adversary $\mathcal{A}$ manifests. In Figure 1, the resulting area A is highlighted by the gray filling between the black borders of the bounding boxes of A . For greater flexibility, R_A^s is implemented as a list to accommodate a wider range of local effects beyond those depicted in Figure 1. The shape depicted in Figure 12b is an abstracted and simplified version of the shape identified in [DLM21] as the shape affected by EMFI when the tip of the coil emitting the electromagnetic field is close to the surface of the targeted IC [DLM21, Fig. 11]. To identify the cells $c_v \in \mathcal{C}_{V_B}$ that are overlapped by an area A , we first use Algorithm 3 to determine which cells $c_v \in \mathcal{C}_{V_B}$ are overlapped by the primary rectangle r_A^p . This involves executing Algorithm 3 for all cells $c_v \in \mathcal{C}_{V_B}$. For each cell $c_v \in \mathcal{C}_{V_B}$ where Algorithm 3 returns **true**, we then check if the bounding box b_{c_v} is fully covered by any of the secondary rectangles $r_A^s \in R_A^s$ of A . This check is performed for each cell c_v and each secondary rectangle $r_A^s \in R_A^s$ using Algorithm 4. If a cell c_v has an overlap with r_A^p but is not fully covered any $r_A^s \in R_A^s$, we add $v \in V_B$ to the set of structural gates V_B^A that the adversary $\mathcal{A}$, targeting area A , can fault. This is implemented by adding the name n_{c_v} of the cell c_v to a set. Since the name of the cell c_v corresponds to the internally used name of the structural gate v we ensure that FIESTA can process the derived set. After collecting the set of structural gates V_B^A , the extension processes a set of clock cycles T during which the adversary can fault any gate $v \in V_B^A$. It computes the set of faultable gates $g \in G_C^A$ by generating all combinations of $t \in T$ and $v \in V_B^A$. These combinations define the gates $g \in G_C^A$ with $g = (v, t)$.

The previously described procedure determines the gates affected by an adversary $\mathcal{A}^\zeta$ targeting area A during the cycles $t \in T$. To automatically generate all adversaries $\mathcal{A}^{\zeta_i} \in \mathcal{A}^{\infty, A_0}$ (where $0 \leq i < n^A$) with the same capabilities but with different areas A_i under attack, FIESTA can shift the bounding boxes defining an initial area A_0 across the

Algorithm 3 HasOverlap.

```

1: Input: Primary Rectangle  $r_A^p$ , Bounding Box  $b_{c_v}$ 
2: Output: has_overlap
3: if  $x_{c_v}^u < x_{r^p}^\ell$  then  $\triangleright$  Cell  $c_v$  is left of  $r^p$ 
4:    $\text{has\_overlap} \leftarrow \text{false}$ 
5: if  $y_{c_v}^u < y_{r^p}^\ell$  then  $\triangleright$  Cell  $c_v$  is above of  $r^p$ 
6:    $\text{has\_overlap} \leftarrow \text{false}$ 
7: if  $x_{r^p}^u < x_{c_v}^\ell$  then  $\triangleright$  Cell  $c_v$  is right of  $r^p$ 
8:    $\text{has\_overlap} \leftarrow \text{false}$ 
9: if  $y_{r^p}^u < y_{c_v}^\ell$  then  $\triangleright$  Cell  $c_v$  is below of  $r^p$ 
10:   $\text{has\_overlap} \leftarrow \text{false}$ 
11:  $\text{has\_overlap} \leftarrow \text{true}$   $\triangleright$  There is an overlap
12: return has_overlap

```

Algorithm 4 FullyCovered.

```

1: Input: Secondary Rectangle  $r_A^s$ , Bounding Box  $b_{c_v}$ 
2: Output: is_covered
3:  $l \leftarrow x_{r^s}^\ell < x_{c_v}^\ell$ 
4:  $b \leftarrow y_{r^s}^\ell < y_{c_v}^\ell$ 
5:  $r \leftarrow x_{r^s}^u > x_{c_v}^u$ 
6:  $t \leftarrow y_{r^s}^u > y_{c_v}^u$ 
7:  $\text{is\_covered} \leftarrow l \wedge b \wedge r \wedge t$   $\triangleright$  Fully Covered iff all four equations are true
8: return is_covered

```

surface of the IC under investigation. The number of different adversaries $n^A = n^{A,x} \cdot n^{A,y}$ to be generated must be specified by providing $n^{A,x}$ and $n^{A,y}$. Based on $n^{A,x}$ and $n^{A,y}$, the distance between the areas is automatically derived. We emphasize that, depending on $n^{A,x}$ and $n^{A,y}$, a varying number of areas may overlap. This allows for modeling arbitrary precision in the positions that can be attacked using practical and local fault injection methods. The extension requires the initial area A_0 as input. A_0 is then shifted by adding the coordinate $(i^x \cdot d^x, i^y \cdot d^y)$ to all bounding boxes that belong to A_0 . This includes the primary rectangle $r_{A_0}^p$, but also all $r_{A_0}^s \in R_{A_0}^s$. Adding the coordinate $(i^x \cdot d^x, i^y \cdot d^y)$ to a bounding box, such as the primary rectangle $r_{A_0}^p$, means that this coordinate is added to both the upper bound (x^u, y^u) and the lower bound (x^ℓ, y^ℓ) of the bounding box. This ensures that all areas A_i with $i \in [0, n^A)$ reflect adversaries with the same capabilities, by maintaining constant length ℓ_y and constant width ℓ_x for all bounding boxes defining A_i . The process of shifting all bounding boxes of an area A is implemented by the function in Line 9 of Algorithm 5. The distances d^x and d^y are the distances between two areas on the x -axis (width), and y -axis (length), respectively. They are computed by dividing the width D^x and length D^y of the bounding box of the entire IC, which is extracted with Gdstk, by the number of adversaries $n^{A,x}$ and $n^{A,y}$, respectively. This results in $d^x = \frac{D^x}{n^{A,x}}$ and $d^y = \frac{D^y}{n^{A,y}}$. The extension generates all $A_i \in A^\infty$ by iterating in an outer loop $i^y \in [0, n^{A,y})$ and in an inner loop $i^x \in [0, n^{A,x})$ as depicted in Algorithm 5. For each of the n^A areas generated with Algorithm 5, the previously described procedure for determining the gates affected by an adversary $\mathcal{A}^{\zeta_i}$ utilizing area A_i is repeated. The extension keeps track of the already computed gate combinations and retains only distinct combinations. In other words, if a resulting set of gate combinations $G_C^{A_j}$ is equal to a previously computed combination $G_C^{A_i}$ with $i < j$ and $0 \leq i, j < n^A$, the extension keeps only $G_C^{A_i}$. The set $\mathcal{G}^{A_0}$ of all distinct gate combinations is then used to derive the set $\{\Phi^{\infty, A_i} : 0 \leq i \leq n^A\}$ of all n^A distinct fault combinations. For that purpose $\mathcal{T}$ must be provided to the extension to

Algorithm 5 Area Generation.

```

1: Input:  $A_0, n^{A,x}, n^{A,y}, d^x, d^y$ 
2: Output:  $A^\infty = \{A_0, \dots, A_{n^A-1}\}$ 
3:  $A^\infty \leftarrow \emptyset$ 
4:  $i \leftarrow 0$ 
5: for all  $i^y \in [0, n^{A,y})$  do
6:   for all  $i^x \in [0, n^{A,x})$  do
7:      $s^x = i_x \cdot d^x$ 
8:      $s^y = i_x \cdot d^y$ 
9:      $A_i = \text{ShiftAllBoundingBoxes}(A_0, s^x, s^y)$ 
10:     $A^\infty \leftarrow A^\infty \cup A_i$ 
11:    $i \leftarrow i + 1$ 
12: return  $A^\infty$ 

```

compute the fault combinations as defined in [Section 2.4](#).

The set of all distinct fault combinations is then submitted to FIESTA, which iteratively computes the advantage for all derived n^A adversaries $\mathcal{A}^{\zeta_i} \in \mathcal{A}^{\infty, A_0}$. Note that FIESTA still requires the specification of φ or $\mathbb{D}\Psi^\infty$, as well as the input distribution $\mathbb{D}X^\infty$, which must be provided by the user.

7 Evaluations using Physical Layout Information

In this section, we present the result of the analysis of different adversaries that can only attack gates in a certain area. To limit the computational complexity of the analysis, we explored the effect of these precise adversaries only against a single S-box of the unprotected **AES-128** implementation. For that purpose, we synthesized and implemented the Canright S-box design [Can05] using the SKY130 cell library with `classic` flow of the open-source toolchain OpenLane 2 [SE20]. The resulting implementation has a size of $74\mu\text{m} \times 84\mu\text{m}$. To derive the adversaries, we utilized the extension of FIESTA presented in [Section 6.2](#). By this, we investigated $n^A = 10 \times 74 \times 84 \times 10 = 621,600$ different positions. For each position, we determine the set of gates, which can be simultaneously affected by shifting an area A_0^1 or A_0^2 to that position. The areas A_0^1 and A_0^2 are defined by a primary rectangle $r_{A_0^1}^p$ and $r_{A_0^2}^p$, respectively. The rectangle $r_{A_0^1}^p$ has dimensions $l_{A_0^1}^x = l_{A_0^1}^y = 5\mu\text{m}$ while $r_{A_0^2}^p$ has dimensions $l_{A_0^2}^x = l_{A_0^2}^y = 25\mu\text{m}$. Neither A_0^1 nor A_0^2 include secondary rectangle, thus $R_{A_0^1}^s = R_{A_0^2}^s = \emptyset$. Both A_0^1 and A_0^2 are instantiated with the center of their primary rectangle on the lower coordinate of the bounding box encompassing the entire implementation of the Canright S-box. These dimensions modeled by A_0^1 and A_0^2 are typically targetable using a single focused laser beam [MDR⁺20, DBC⁺18]. Out of the $n^A = 621,600$ positions to which A_0^1 and A_0^2 are shifted, only 528 and 1175 distinct gate combinations were derived, respectively. Assuming each adversary is associated with one position and can induce faults at that position during all clock cycles, we obtain $n^{\mathcal{A}^1} = 528$ and $n^{\mathcal{A}^2} = 1175$ distinct adversaries. From these distinct gate combinations, FIESTA estimated the strongest adversary $A^{\max}$ under the assumption that $\mathcal{T} = \{\tau^{\text{reset}}\}$, and computed its advantage for varying fault probabilities φ . [Table 3](#) illustrates $A^{\max}$ for $\mathcal{A}^\infty$ generated based on A_0^1 and A_0^2 .

From the case studies covered in [Table 3](#), it is evident that, regardless of the targeted gate set, a higher probability of independent faults φ consistently leads to a greater adversarial advantage. Additionally, adversaries utilizing A_0^1 have a higher spatial resolution compared to those utilizing A_0^2 , resulting in a smaller number of gates being affected simultaneously. However, this increased precision corresponds to a reduced advantage across all tested values of φ . While increased precision reduces the adversary's advantage under the general

Table 3: Comparison of adversaries targeting the base area A_0^1 and A_0^2 shifted to different positions and with $\varphi \in [0.125, 0.25, 0.5]$.

Adversaries	φ	n^A	$Adv(\mathcal{A}^{\max})$	
			μ^ℓ	μ^u
$\mathcal{A}^{\infty, A_0^1}$	0.125	528	0.46932689	0.47373787
	0.250		0.71092235	0.71491997
	0.500		0.90471207	0.90729109
$\mathcal{A}^{\infty, A_0^2}$	0.125	1175	0.55786611	0.56225228
	0.250		0.80387247	0.80736937
	0.500		0.95529246	0.95710136

random fault model, resulting in fewer effective faults, we note that for certain FI attacks, such as DFA, knowledge of the precise fault locations is a necessary condition. Therefore, analyzing even more precise adversaries remains meaningful, even if their advantage is reduced.

Additionally, the extension presented in [Section 6](#) not only allows access to different gate-level implementations that utilize various algorithms to protect against FI, which is already possible with FIESTA, but it also enables the assessment of the security of different physical-level implementations. This means that using this extension, it is possible to automatically compare the effects of place-and-route strategies and determine whether incorporating physical properties of a circuit can achieve a higher protection level against FI. This is particularly relevant for highly precise adversaries with high spatial resolution, which pose a significant threat to ICs due to the requirement of some efficient FI attacks for high precision in injected faults, as previously mentioned. We leave such explorations for future work, which can be combined with the development of an automated strategy to implement efficiently protected ICs that utilize physical properties of the circuit, such as the placement of different gates.

8 Conclusion

In this work, we introduced FIESTA, an automated framework for the statistical evaluation of fault adversaries. FIESTA enables the assessment of large-scale hardware designs against (general) random fault adversaries, providing a high level of confidence in the results. By leveraging a non-exhaustive, statistical approach, FIESTA can handle much larger designs compared to existing state-of-the-art tools, while maintaining a reasonable level of confidence in the evaluation. We demonstrated how abstract fault adversaries, defined by the (general) random fault model, can be evaluated using FIESTA. Additionally, we proposed an approach to accurately assess the impact of practical adversaries, such as those utilizing laser fault injection or EMFI, on the physical design implementation. This approach involves refining abstract and generic fault models based on insights gained from practical experiments. Using FIESTA, we evaluated several AES designs protected by different countermeasures. Our results demonstrate that the guarantees provided by certain state-of-the-art countermeasures, such as protection against a limited number of faults, can also provide an increased level of protection against multiple randomly injected faults. However, in line with practical observations, our case studies revealed that simple replication and implementation of a majority vote do not provide the expected level of protection. Instead, only countermeasures that effectively prevent fault propagation result in a decent level of protection. This insight underscores the importance of carefully evaluating and selecting countermeasures to ensure robust protection against fault injection attacks in cryptographic hardware. By bridging the gap between the evaluation of gate-

level (algorithmic countermeasures) and physical-level (place-and-route) countermeasures, FIESTA enables the study of the effects of a new class of countermeasures. This approach provides a more comprehensive assessment of security measures in hardware design, integrating both algorithmic strategies and physical design considerations to enhance overall protection.

Acknowledgments

The work described in this paper has been supported in part by the German Research Foundation (DFG) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972, and by the Federal Ministry of Education and Research of Germany through the Project KOSEF (16KIS1597).

References

- [AMR⁺20] Anita Aghaie, Amir Moradi, Shahram Rasoolzadeh, Aein Rezaei Shahmirzadi, Falk Schellenberg, and Tobias Schneider. Impeccable Circuits. *IEEE Trans. Computers*, 69(3):361–376, 2020.
- [AWMN20] Victor Arribas, Felix Wegener, Amir Moradi, and Svetla Nikova. Cryptographic Fault Diagnosis using VerFI. In *IEEE HOST 2020*, pages 229–240. IEEE, 2020.
- [BBM⁺22] Timo Bartkewitz, Sven Bettendorf, Thorben Moos, Amir Moradi, and Falk Schellenberg. Beware of Insufficient Redundancy An Experimental Evaluation of Code-based FI Countermeasures. *IACR TCHES*, 2022(3):438–462, 2022.
- [BDL97] Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. On the Importance of Checking Cryptographic Protocols for Faults (Extended Abstract). In *EUROCRYPT 1997*, volume 1233 of *LNCS*, pages 37–51. Springer, 1997.
- [BFG⁺24] Sonia Belaïd, Jakob Feldtkeller, Tim Güneysu, Anna Guinet, Jan Richter-Brockmann, Matthieu Rivain, Pascal Sasdrich, and Abdul Rahman Taleb. Formal Definition and Verification for Combined Random Fault and Random Probing Security. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VII*, volume 15490 of *LNCS*, pages 167–200. Springer, 2024.
- [BMRT22] Sonia Belaïd, Darius Mercadier, Matthieu Rivain, and Abdul Rahman Taleb. IronMask: Versatile Verification of Masking Security. In *SP 2022*, pages 142–160. IEEE, 2022.
- [Boo15] Boost. Boost C++ Libraries. <http://www.boost.org/>, 2015. Last accessed 2025-01-15.
- [BS97] Eli Biham and Adi Shamir. Differential Fault Analysis of Secret Key Cryptosystems. In *CRYPTO 1997*, volume 1294 of *LNCS*, pages 513–525. Springer, 1997.
- [Can05] David Canright. A Very Compact S-Box for AES. In *CHES 2005*, volume 3659 of *LNCS*, pages 441–455. Springer, 2005.

- [Cla07] Christophe Clavier. Secret External Encodings Do Not Prevent Transient Fault Analysis. In *CHES 2007*, volume 4727 of *LNCS*, pages 181–194. Springer, 2007.
- [CS21] Gaëtan Cassiers and François-Xavier Standaert. Provably Secure Hardware Masking in the Transition- and Glitch-Robust Probing Model: Better Safe than Sorry. *IACR TCHES*, 2021(2):136–158, 2021.
- [DBC⁺18] Jean-Max Dutertre, Vincent Beroulle, Philippe Candelier, Stephan De Castro, Louis-Barthelemy Faber, Marie-Lise Flottes, Philippe Gendrier, David Hély, Régis Leveugle, Paolo Maistri, Giorgio Di Natale, Athanasios Papadimitriou, and Bruno Rouzeyre. Laser Fault Injection at the CMOS 28 nm Technology Node: an Analysis of the Fault Model. In *2018 Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTC 2018, Amsterdam, The Netherlands, September 13, 2018*, pages 1–6. IEEE Computer Society, 2018.
- [DEK⁺18] Christoph Dobraunig, Maria Eichlseder, Thomas Korak, Stefan Mangard, Florian Mendel, and Robert Primas. SIFA: Exploiting Ineffective Fault Inductions on Symmetric Cryptography. *IACR TCHES 2018*, 2018(3):547–572, 2018.
- [DLM19] Mathieu Dumont, Mathieu Lisart, and Philippe Maurine. Electromagnetic Fault Injection : How Faults Occur. In *FDTC 2019*, pages 9–16. IEEE, 2019.
- [DLM21] Mathieu Dumont, Mathieu Lisart, and Philippe Maurine. Modeling and Simulating Electromagnetic Fault Injection. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, 40(4):680–693, 2021.
- [DN20] Siemen Dhooghe and Svetla Nikova. My Gadget Just Cares for Me - How NINA Can Prove Security Against Combined Attacks. In Stanislaw Jarecki, editor, *Topics in Cryptology - CT-RSA 2020 - The Cryptographers’ Track at the RSA Conference 2020, San Francisco, CA, USA, February 24–28, 2020, Proceedings*, volume 12006 of *LNCS*, pages 35–55. Springer, 2020.
- [DN23] Siemen Dhooghe and Svetla Nikova. The Random Fault Model. In *SAC 2023*, volume 14201 of *LNCS*, pages 191–212. Springer, 2023.
- [DR02] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Information Security and Cryptography. Springer, 2002.
- [FGS23] Jakob Feldtkeller, Tim Güneysu, and Patrick Schaumont. *Quantitative Fault Injection Analysis*, pages 302–336. Springer Nature Singapore, 12 2023.
- [FJLT13] Thomas Fuhr, Éliane Jaulmes, Victor Lomné, and Adrian Thillard. Fault Attacks on AES with Faulty Ciphertexts Only. In *FTDC 2013*, pages 108–118. IEEE Computer Society, 2013.
- [Gab20] Lucas H. Gabrielli. Gdstk (GDSII Tool Kit). <https://heitzmann.github.io/gdstk/>, 2020. Accessed: 2025-04-14.
- [GYG⁺18] Marjan Ghodrati, Bilgiday Yuce, Surabhi Gujar, Chinmay Deshpande, Leyla Nazhandali, and Patrick Schaumont. Inducing local timing fault through EM injection. In *Proceedings of the 55th Annual Design Automation Conference, DAC 2018, San Francisco, CA, USA, June 24–29, 2018*, pages 142:1–142:6. ACM, 2018.

- [HCP⁺24] Vedad Hadzic, Gaëtan Cassiers, Robert Primas, Stefan Mangard, and Roderick Bloem. Quantile: Quantifying Information Leakage. *IACR TCHES*, 2024(1):433–456, 2024.
- [IPSW06] Yuval Ishai, Manoj Prabhakaran, Amit Sahai, and David Wagner. *Private Circuits II: Keeping Secrets in Tamperable Circuits*, pages 308–327. Springer Berlin Heidelberg, 2006.
- [KM22] David Knichel and Amir Moradi. Low-Latency Hardware Private Circuits. In *CCS 2022*, pages 1799–1812. ACM, 2022.
- [KMMS22] David Knichel, Amir Moradi, Nicolai Müller, and Pascal Sasdrich. Automated Generation of Masked Hardware. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):589–629, 2022.
- [LSG⁺10] Yang Li, Kazuo Sakiyama, Shigeto Gomisawa, Toshinori Fukunaga, Junko Takahashi, and Kazuo Ohta. Fault Sensitivity Analysis. In *CHES 2010*, volume 6225 of *LNCS*, pages 320–334. Springer, 2010.
- [MDR⁺20] Alexandre Menu, Jean-Max Dutertre, Jean-Baptiste Rigaud, Brice Colombier, Pierre-Alain Moëllic, and Jean-Luc Danger. Single-bit Laser Fault Model in NOR Flash Memories: Analysis and Exploitation. In *17th Workshop on Fault Detection and Tolerance in Cryptography, FDTC 2020, Milan, Italy, September 13, 2020*, pages 41–48. IEEE, 2020.
- [MM22] Nicolai Müller and Amir Moradi. PROLEAD A Probing-Based Hardware Leakage Detection Tool. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):311–348, 2022.
- [MM24] Nicolai Müller and Amir Moradi. Automated Generation of Fault-Resistant Circuits. *IACR TCHES*, 2024(3):136–173, 2024.
- [MN98] Makoto Matsumoto and Takuji Nishimura. Mersenne twister. *ACM Transactions on Modeling and Computer Simulation*, 8:3–30, 1 1998.
- [NDR⁺23] Roukoz Nabhan, Jean-Max Dutertre, Jean-Baptiste Rigaud, Jean-Luc Danger, and Laurent Sauvage. A Tale of Two Models: Discussing the Timing and Sampling EM Fault Injection Models. In *Workshop on Fault Detection and Tolerance in Cryptography, FDTC 2023, Prague, Czech Republic, September 10, 2023*, pages 1–12. IEEE, 2023.
- [OGM17] Sébastien Ordas, Ludovic Guillaume-Sage, and Philippe Maurine. Electromagnetic fault injection: the curse of flip-flops. *J. Cryptogr. Eng.*, 7(3):183–197, 2017.
- [QS01] Jean-Jacques Quisquater and David Samyde. ElectroMagnetic Analysis (EMA): Measures and Counter-Measures for Smart Cards. In *E-smart 2001*, volume 2140 of *LNCS*, pages 200–210. Springer, 2001.
- [RFSG22] Jan Richter-Brockmann, Jakob Feldtkeller, Pascal Sasdrich, and Tim Güneysu. VERICA - Verification of Combined Attacks Automated formal verification of security against simultaneous information leakage and tampering. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):255–284, 2022.
- [RSG23] Jan Richter-Brockmann, Pascal Sasdrich, and Tim Güneysu. Revisiting Fault Adversary Models - Hardware Faults in Theory and Practice. *IEEE Trans. Computers*, 72(2):572–585, 2023.

- [RSM21] Shahram Rasoolzadeh, Aein Rezaei Shahmirzadi, and Amir Moradi. Impeccable Circuits III. In *IEEE ITC 2021*, pages 163–169. IEEE, 2021.
- [RSS⁺21] Jan Richter-Brockmann, Aein Rezaei Shahmirzadi, Pascal Sasdrich, Amir Moradi, and Tim Güneysu. FIVER - Robust Verification of Countermeasures against Fault Injections. *IACR TCHES*, 2021(4):447–473, 2021.
- [SA02] Sergei P. Skorobogatov and Ross J. Anderson. Optical Fault Induction Attacks. In *CHES 2002*, volume 2523 of *LNCSS*, pages 2–12. Springer, 2002.
- [SAB⁺23] Sayandeep Saha, Manaar Alam, Arnab Bag, Debdeep Mukhopadhyay, and Pallab Dasgupta. Learn from Your Faults: Leakage Assessment in Fault Attacks Using Deep Learning. *Journal of Cryptology*, 36, 7 2023.
- [Sch08] Fritz Scholz. Confidence Bounds & Intervals for Parameters Relating to the Binomial, Negative Binomial, Poisson and Hypergeometric Distributions - With Applications to Rare Events, 11 2008.
- [SE20] Mohamed Shalan and Tim Edwards. Building OpenLANE: A 130nm OpenROAD-based Tapeout- Proven Flow : Invited Paper. In *IEEE/ACM ICCAD 2020*, pages 110:1–110:6. IEEE, 2020.
- [SRM20] Aein Rezaei Shahmirzadi, Shahram Rasoolzadeh, and Amir Moradi. Impeccable Circuits II. In *ACM/IEEE DAC 2020*, pages 1–6. IEEE, 2020.
- [WA08] Fan Wang and Vishwani D. Agrawal. Single Event Upset: An Embedded Tutorial. In *21st International Conference on VLSI Design (VLSI Design 2008)*, 4-8 January 2008, Hyderabad, India, pages 429–434. IEEE Computer Society, 2008.
- [Wol] Claire Wolf. Yosys Open SYnthesis Suite. <https://yosyshq.net/yosys/>.